

Interim Report

Level 4

Post-Hoc Formal Verification of LLM-Generated Python Code Using BPMN-Derived Temporal Specifications and Hybrid Parsing

Group 18 - Epsilon

Index Number	Name
214063M	FERNANDO WKD
214248K	WELMILLA CN
214077J	HETTIARACHCHI HTV

Faculty of Information Technology

University of Moratuwa

2026

Interim Report

Level 4

Post-Hoc Formal Verification of LLM-Generated Python Code Using BPMN-Derived Temporal Specifications and Hybrid Parsing

Group 18 - Epsilon

Index Number	Name
214063M	FERNANDO WKD
214248K	WELMILLA CN
214077J	HETTIARACHCHI HTV

Supervised by: Dr. Thilina Thanthriwatta

Faculty of Information Technology

University of Moratuwa

2026

Abstract

The rapid adoption of Large Language Models (LLMs) for software generation has introduced a critical verification gap. Code that appears syntactically correct may contain subtle semantic flaws invisible to conventional testing. This is particularly acute in state based workflow logic, where business processes must adhere to strict temporal and structural constraints. This interim report presents the design and early implementation of VibeCheck, a verified translation validation framework that formally verifies LLM-generated Python workflow implementations against their originating Business Process Model and Notation (BPMN) specifications.

The proposed framework contains a verification architecture that independently constructs formal models from both the BPMN specification and the generated Python code, then systematically compares these models to detect conformance violations. The specification track extracts temporal logic properties with provable coverage guarantees, while the code track extracts and validates an intermediate representation through multi-modal translation validation combining differential execution, symbolic refinement checking, and structural correspondence verification. The two tracks converge in a process level equivalence analysis that distinguishes genuine conformance violations from acceptable implementation variations.

The early implementation has the FLOW-BENCH dataset for ground-truth workflow specifications, complemented by a synthetic data generation protocol that injects controlled defects to validate the verification pipeline itself. The framework is implemented using Python's ast module for dynamic instrumentation, the Z3 theorem prover for symbolic refinement, and the SPOT model checking library for automata theoretic verification.

Table of Contents

Chapter 1 - Introduction	
1.1 Introduction	vi
1.2 Background and Motivation	vi
1.3 Problem in Brief.....	vii
1.4 Aim and Objectives.....	viii
1.5 Proposed Solution	ix
Chapter 2 - Literature Review	xi
2.1 Business Process verification	xi
2.2 Generative AI in Software Engineering	xiii
2.3 Translation Validation and Equivalence Checking	xv
2.4 Comparison of Approaches	xx
Chapter 3 - Technology Adapted	xxii
3.1 Python and the ast Module	xxii
3.2 Dynamic Tracing with sys.settrace	xxii
3.3 Z3 Theorem Prover for Symbolic Refinement	xxiii
3.4 SPOT Model Checking Library	xxiv
3.5 NetworkX for Graph Analysis.....	xxiv
3.6 Version Control and Collaboration Tools	xxv
Chapter 4 - Our Approach: Verified Translation Validation	xxvi
4.1 Dual-Track Architecture Overview	xxvi
4.2 Module 01 : Property Extraction with Coverage Guarantees.....	xxviii
4.3 Module 02: Verified IR Extraction	xxix
4.4 Module 03: Equivalence Analysis	xxxi
Chapter 5 - Analysis and Design.....	xxxv

5.1 System Architecture Overview	xxxv
5.2 Module Descriptions	xxxv
5.3 Data Flow and Interactions.....	xxxvii
Chapter 6 - Implementation	xxxix
6.1 Dataset Preparation	xxxix
6.2 Implementation	xxxix
Chapter 7 - Discussion.....	xliii
7.1 Interim Test Results	xliii
7.2 Summary of the Report	xlvi
7.3 Novelty and Differentiation.....	xlviii
7.4 Present Plan on Further Work	xlix
References	l
Appendix A - Individual's Contribution to the Project.....	liii

List of Figures

Figure 2.1 - Structure of the BPMN to Petri net transformation tool	12
Figure 2.2 - Global schema of model checking approach for detecting structural errors	12
Figure 2.3 - FLOWBENCH Overview	14
Figure 2.4 - Concept of Translation Validation	16
Figure 2.5 - A generic clone detection process	19
Figure 4.1 - Dual-Track Verification Architecture	27
Figure 5.1 - Top-Level System Architecture Diagram	35
Figure 5.2 - Module Interaction and Data Flow	37
Figure 7.1 - Terminal output semantic mapping of BPMN sequence flows and conditional logic into a json intermediate representation	43
Figure 7.2 - Code Structure of AST parser	44
Figure 7.3 - Terminal output demonstrating the successful AST parsing and JSON-based WIR extraction for both standard and anomalous Python scripts	45
Figure 7.4 - main_role_c.py file	46
Figure 7.5 - Execution output demonstrating the deterministic lifting of JSON intermediate representations into formal Labeled Transition Systems (LTS), preserving action labels and guard conditions.	47

List of Tables

Table 2.1 - Comparison of Related Approaches	20
Table 4.1 - BPMN Pattern-to-Property Mapping	29
Table 4.2 - Validation Modes and Confidence Types	30
Table 4.3 - Equivalence Level Definitions	31
Table 5.1 - System Modules and Their Functions	36

Chapter 1 - Introduction

1.1 Introduction

1.2 Background and Motivation

The software engineering landscape is currently undergoing a considerable shift of a large scale, characterized by the rapid adoption of Large Language Models (LLMs) such as GPT-4, Claude 3.5, and DeepSeek-Coder. These models have popularized "vibe coding," a development style where non-expert users generate entire software modules from natural language prompts [1]. While these stochastic models demonstrate remarkable proficiency in syntactic generation, producing code that appears structurally correct and often passes standard unit tests, they operate fundamentally as probabilistic token predictors rather than logical reasoning engines.

This distinction creates what researchers have termed the "Verification Crisis in Generative Software Engineering" [2]. LLM-generated software frequently exhibits syntactic perfection while harboring subtle semantic flaws, safety violations, and logic errors that evade conventional testing paradigms. In high-assurance domains such as financial services, healthcare administration, and automated control systems, functional validation alone is insufficient. The stochastic nature of LLM generation introduces the risk of "hallucinations" in logic, where a generated workflow might satisfy primary functional objectives while violating critical negative constraints such as "never approve a loan without a credit check" or "inventory must never be negative."

Current benchmarks for evaluating LLM code generation, including HumanEval and MBPP, focus primarily on stateless, algorithmic tasks such as sorting lists or string manipulation [3]. However, real-world business applications are inherently stateful. A vending machine controller, a loan approval pipeline, or an inventory management system maintains internal state over time, and violations of state invariants can have severe financial and safety implications. Recent research utilizing workflow-specific benchmarks has demonstrated that LLM-driven code generation frequently loses

coherence in long-horizon simulations, exhibiting state drift errors that are invisible to standard unit tests [4].

Formal verification tools such as NuSMV (New Symbolic Model Verifier) and SPOT offer powerful mechanisms for exhaustively exploring state spaces to prove that no invalid state is ever reachable [5]. However, these tools remain inaccessible to the average developer, requiring specifications to be written in complex temporal logics and models to be defined in specialized description languages. There is currently no automated framework that bridges the gap between natural language requirements, LLM-generated Python code, and rigorous mathematical proof of correctness. This project addresses precisely this gap.

1.3 Problem in Brief

There is no automated mechanism to mathematically guarantee that LLM-generated workflow code adheres to business rules.

1. **The Black Box Problem:** Users treat LLMs as non-transparent generators, trusting output that appears syntactically correct but may contain hidden logic flaws.
2. **The Hallucination Risk:** LLMs act as probabilistic engines, not logical ones. They frequently hallucinate state transitions that violate strict business constraints.
3. **The Verification Gap:** Existing formal verification tools (NuSMV) require expert level knowledge. There is no automated framework to bridge the gap between natural language prompts, LLM generated Python code, and heavy mathematical proof.

This project addresses the lack of a “Post-Hoc Auditor” that can independently verify the semantic correctness of state-based Python workflows generated by AI.

1.4 Aim and Objectives

The primary aim of this research is to develop a verified translation validation framework that enables trustworthy, automated verification of LLM-generated Python workflow implementations against their originating Business Process Model and Notation (BPMN) specifications. The framework specifically targets finite business processes with bounded iteration, where the semantic gap between visual process models and executable code is most pronounced, and where multiple semantically distinct implementations may satisfy the same functional requirements.

Rather than attempting to constrain the LLM during generation, this framework adopts a post-hoc verification strategy that accepts the probabilistic nature of LLM outputs and subjects them to rigorous deterministic audit. The central contribution is a dual-track architecture that independently constructs formal models from both the specification (BPMN) and the implementation (Python code), then systematically compares these models to detect process-level conformance violations that functional testing cannot identify.

Objectives

To achieve this aim, the project is divided into the following specific objectives:

- Objective 1 (Specification Extraction): To automatically parse BPMN 2.0 XML specifications and construct a coverage guaranteed semantic map, which gives a formal automaton (M_{spec}) that represents the business rules.
- Objective 2 (Verified Translation Validation): To translate raw LLM-generated Python scripts into validated Workflow Intermediate Representations (WIR) using dynamic trace collection and the Z3 Theorem Prover, successfully filtering out logical hallucinations and unsafe syntax.
- Objective 3 (Process Equivalence and Clustering): To lift the JSON-based WIRs into C++ mathematical memory spaces using the SPOT library, and to develop a stuttering bisimulation algorithm that mathematically clusters functionally equivalent but structurally divergent AI implementations.
- Objective 4 (Formal Model Checking): To execute an automata-theoretic intersection between the specification automaton (M_{spec}) and the code-

derived representative automaton (M_{code}), issuing a deterministic pass/fail verdict and classifying process anomalies.

1.5 Proposed Solution

The proposed solution, VibeCheck, is a validation engine designed to act as a mathematical firewall between AI generation and production deployment.

- **Target Users:** The system is designed for enterprise software architects, Quality Assurance (QA) engineers, and business analysts who use Large Language Models to generate workflow code but require deterministic proof of its safety and compliance.
- **Input:** The system takes in standard BPMN 2.0 XML files paired with Natural Language specifications (such as those from the IBM FLOW-BENCH dataset), alongside a batch of varying LLM-generated Python implementations attempting to fulfill those specifications.
- **Process:** The framework operates across three distinct domains. First, it extracts mathematical constraints from the visual BPMN model. Concurrently, it extracts a syntactic and symbolic Intermediate Representation (WIR) from the AI-generated Python code. Finally, it uses graph reduction (Stuttering Bisimulation) to strip away the AI's "silent" non-impactful steps, allowing for a pure mathematical cross-check between the code's behavior and the business rules.
- **Output:** Rather than probabilistic test scores, the system outputs a definitive Conformance Matrix. It provides a deterministic verdict (System Passed/Failed) and a classified breakdown of discrepancies if the code fails.
- **Technology Stack:** The solution integrates Python and Abstract Syntax Tree (AST) parsing. It utilizes the Microsoft Z3 engine for symbolic refinement, and the high-performance C++11 SPOT library, powered by the BuDDy Binary Decision Diagram (BDD) dictionary, for complex automata manipulation and model checking.
- **System Requirements:** Due to the low-level memory requirements of the SPOT library and graph traversal operations, the system requires a Linux-based environment (or Windows Subsystem for Linux - WSL), Python 3.8 or

higher, and a modern multi-core processor to handle the computational overhead of state-space exploration.

1.6 Structure of the Dissertation

The remainder of this interim report is organized as follows. Chapter 2 presents a comprehensive literature review, including traditional BPMN-to-Petri-net translation approaches, temporal logic verification techniques, LLM code generation evaluation methodologies, and translation validation strategies from the verified compilation literature. Chapter 3 describes the key technologies adapted for this project and explains their use for the verification task. Chapter 4 presents our novel approach, the dual-track verified translation validation architecture, detailing the three roles and their respective methodologies. Chapter 5 describes the system analysis and design, including the top-level architecture diagram and module interactions. Chapter 6 reports on early implementation steps. Chapter 7 provides a discussion of the work completed to date, the novelty of our contribution, and the plan for further evaluation. Appendix A documents each team member's individual contribution to the project.

Chapter 2 - Literature Review

2.1 Business Process verification

2.1.1 Overview

The formal verification of business processes has historically focused on ensuring that visual models conform to predefined constraints. With the advancement of Large Language Models (LLMs) in software engineering, the verification landscape must now bridge the gap between high-level specifications and code generation. This review synthesizes foundational approaches in process verification, evaluates the current state of LLM code generation, and explores translation validation paradigms to establish the theoretical foundation for the VibeCheck framework.

2.1.2 Traditional BPMN-to-Petri-Net Translation

The formal verification of business processes has historically relied on translating BPMN diagrams into Petri nets, a well-established formalism with rich analysis tools. Dijkman et al. [6] proposed one of the foundational approaches in this space, defining a formal semantics for BPMN through a systematic mapping to Petri nets. Their work established that BPMN constructs including tasks, gateways, and events could be represented as Petri net elements, enabling the use of existing verification tools to check properties such as deadlock freedom and liveness.

This approach has several important characteristics. First, it leverages the mature ecosystem of Petri net analysis tools, providing access to decades of research in model checking and structural analysis. Second, the translation is deterministic and well-defined, ensuring that the resulting Petri net faithfully represents the BPMN semantics. Third, verification results on the Petri net can be traced back to the original BPMN diagram, providing meaningful diagnostic information to process designers.

However, the BPMN-to-Petri-net approach has significant limitations in the context of LLM-generated code verification. Most critically, it operates as a static specification check; while the Petri net can prove the BPMN model is sound, it lacks a

mechanism to ingest and audit the non-deterministic execution traces of Python implementations. Furthermore, Petri-net formalisms are primarily designed for Reachability Analysis rather than the Temporal Ordering of business logic. This creates a 'verification gap' where the state-based logic of the Petri net cannot easily account for the data-driven hallucinations common in LLM code. Our work explicitly addresses this by shifting from state-transition models to Metric-Driven LTLf Extraction, enabling a nuanced, trace-level audit with explicit coverage guarantees.

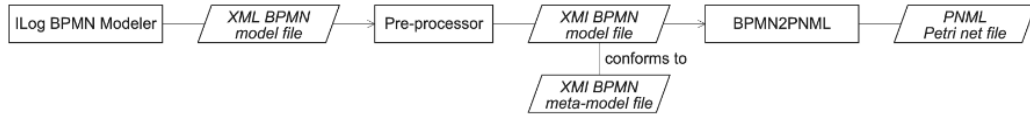


Figure 2.1 Structure of the BPMN to Petri net transformation tool.

2.1.3 Temporal Logic Verification of Business Processes

Kherbouche and Ahmad [7] advanced the field by proposing the use of Linear Temporal Logic (LTL) property patterns for BPMN verification, utilizing the SPIN model checker for automated verification. Their work demonstrated that common process constraints such as ordering, mutual exclusion, and eventual completion could be expressed as LTL formulas and automatically verified against process models.

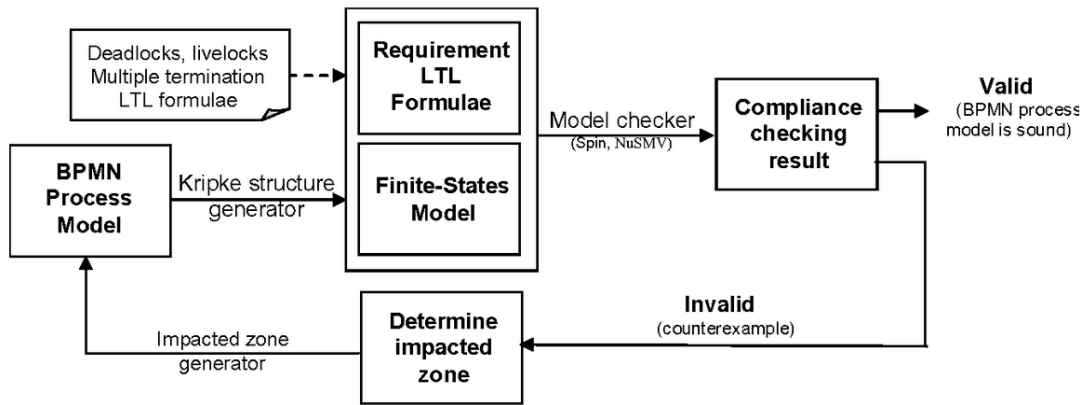


Figure 2.2 Global schema of model checking approach for detecting structural errors

The YAWL-to-FSP translation work [8] extended these ideas by applying fluent temporal logic to workflow verification, providing a more expressive property specification language.

De Giacomo and Vardi [9] addressed a fundamental limitation of standard LTL by introducing Linear Temporal Logic on Finite Traces (LTLf), which is specifically designed for finite executions such as business processes. Since standard LTL assumes infinite traces, it cannot directly express properties about process termination or final state constraints. LTLf resolves this by providing temporal operators that are interpreted over finite sequences, making it the appropriate logic for business process verification. Giacomo, Masellis and Montali [10] further explored the relationship between LTLf and standard LTL, establishing the theoretical foundations for the translation between these logics.

Despite these advances, existing approaches share a common limitation. They verify the BPMN model in isolation, without reference to any implementation. The properties are checked against the process model derived from the diagram, not against the executable code that purports to implement that process. Our framework bridges this gap by verifying the implementation against the specification, a fundamentally different verification target.

2.2 Generative AI in Software Engineering

2.2.1 LLM Code Generation Benchmarks

The evaluation of LLM-generated code has evolved significantly in recent years, shifting from isolated algorithmic tasks to complex, state-based software engineering challenges. Early benchmarks in the field, most notably OpenAI’s HumanEval and the Mostly Basic Python Problems (MBPP) dataset. They focused almost exclusively on standalone, stateless function generation. These benchmarks evaluate models using the *pass@k* metric against straightforward unit tests. However, researchers quickly realized that success on HumanEval did not translate to real world software engineering, where code must maintain state, handle multi-step logic, and interact with external dependencies.

To bridge this gap, Du et al. [4] introduced ClassEval, a manually-crafted benchmark designed to evaluate LLMs on class level, object oriented code generation. ClassEval demonstrated that while LLMs excel at writing single functions, their performance degrades significantly when required to maintain context across multiple

interdependent class methods, manage internal object states, or utilize inheritance. By forcing the LLM to orchestrate logic across a broader architectural scope, ClassEval proved that existing benchmarks were vastly insufficient for assessing complex code generation tasks.

However, even class level evaluation falls short of capturing the strict procedural constraints required in enterprise business applications. To address this, the FLOW-BENCH dataset [11] extended evaluation directly into the domain of workflow specific code generation. FLOW-BENCH is uniquely critical to the field of process verification because it moves beyond generic Python scripting to evaluate an LLM's ability to strictly adhere to business logic constraints. The dataset provides paired triads of data: natural language descriptions, formal BPMN 2.0 XML diagrams, and their corresponding Python implementations. By incorporating realistic business control flow patterns, such as exclusive (XOR) decision gateways, parallel (AND) execution branches, and bounded loops, FLOW-BENCH serves as a rigorous ground truth for verification research. It allows researchers to move beyond simple functional testing and mathematically evaluate whether an AI-generated script successfully preserves the structural and temporal constraints of an originating business specification.

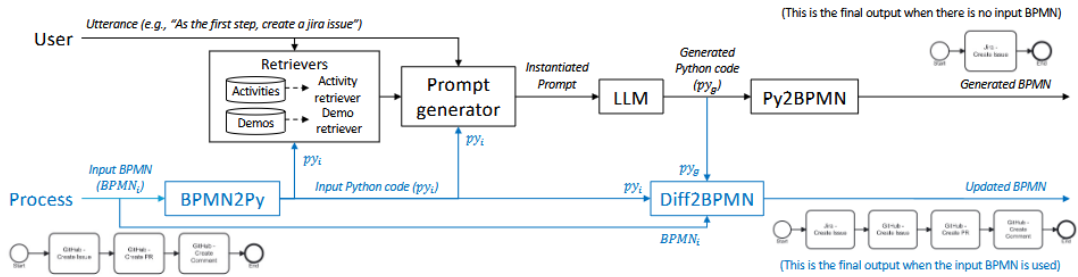


Figure 2.3 FLOWBENCH Overview

2.2.2 The Verification Crisis and Logic Hallucinations

Li et al. [12] conducted a comprehensive survey of hallucination risks in LLM-generated code, identifying categories of errors including logic errors, API misuse, and constraint violations. Their analysis highlighted the insufficiency of traditional testing approaches for detecting subtle semantic errors in generated code. EvalPlus [13] proposed rigorous functional testing through expanded test suites, but

acknowledged the fundamental limitation of testing: it can demonstrate the presence of bugs but cannot prove their absence.

The Vending-Bench dataset [14] provided compelling evidence of the "Meltdown" phenomenon, where LLM-driven agents lose coherence in long-horizon stateful simulations, hallucinating inventory states or allowing transactions that violate business constraints. These studies collectively motivate the need for formal verification approaches that can provide mathematical guarantees beyond what testing alone can achieve.

2.2.3 Static Analysis and Intermediate Representation

To systematically detect the semantic errors and logic hallucinations present in LLM-generated software, the raw source code must be translated into a mathematically analyzable format. Traditional static analysis tools operate directly on source text, but dynamically typed languages like Python present significant challenges for simple lexical scanning. To resolve this, modern program analysis relies on constructing Abstract Syntax Trees (AST). As outlined by Binkley [35], ASTs strip away syntactic sugar to represent the hierarchical, structural logic of the program.

However, raw ASTs are often too granular for workflow verification. To bridge the gap between code structure and business logic, compilers and verification engines utilize Intermediate Representations (IR). In the context of business process modeling, extracting a Workflow Intermediate Representation (WIR) is critical. A WIR abstracts standard programming constructs (such as variable assignments or basic loops) into distinct state-transition nodes. This extraction process, as utilized in advanced translation validation pipelines, isolates the observable control-flow graph from the underlying implementation details, preparing the untrusted code for formal constraint checking.

2.3 Translation Validation and Equivalence Checking

2.3.1 Verified Compilation Principles

The concept of translation validation originates from the verified compilation literature, most notably the CompCert project [15]. Rather than proving the compiler

correct once and for all, translation validation proves that each individual compilation produces a target program that is semantically equivalent to the source program. This approach decouples the correctness of individual translations from the correctness of the translation tool itself, a principle that directly informs our design.

Cordeiro and Fischer [16] applied translation validation to software model checking, demonstrating that program transformations in verification tools could be validated post hoc rather than verified a priori. This work established that translation validation is both practical and effective for complex program analysis pipelines. However, these approaches assume a trusted source program and a deterministic translation process. In our setting, the source (LLM-generated code) is inherently untrusted, and the extraction process involves significant semantic abstraction.

2.3.2 Process Equivalence

Pnueli et al. [17] pioneered the translation validation paradigm, formalizing it through the concept of a *simulation relation*. In classical formal methods, a simulation relation provides a mathematical guarantee that every observable behavior of a target program (the generated code) is explicitly permitted by the source program (the specification). If the generated model successfully simulates the source model, it proves that the target has not invented any unauthorized behaviors.

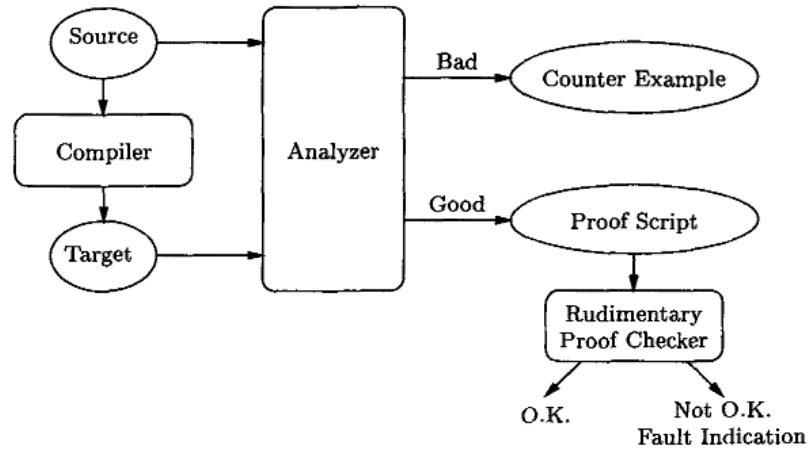


Figure 2.4 Concept of Translation Validation

However, applying classical equivalence relations to LLM-generated software introduces a severe theoretical hurdle. Due to their nature, Large Language Models do

not generate uniform implementations. When tasked with translating a BPMN workflow into Python, an LLM will inevitably introduce a wide array of structurally diverse but functionally benign operations.

2.3.3 Algorithmic Approaches to Stuttering Bisimulation

While the theoretical concept of stuttering bisimulation resolves the abstraction of silent transitions, computing this equivalence efficiently on large generated codebases requires advanced graph reduction algorithms. Baier and Katoen [26], in their foundational work on model checking, established that process equivalence can be computed by reducing transition systems into minimized quotient graphs.

To achieve this programmatically, modern equivalence engines rely on partition refinement algorithms, originally pioneered by Paige and Tarjan [27] and later adapted by Groote and Vaandrager [28] specifically for branching and stuttering bisimulation. These algorithms operate by iteratively partitioning the state space into equivalence classes. If an LLM introduces extraneous states that do not transition to a differing observable business state, the refinement algorithm mathematically merges them into a single equivalence class. This literature underscores that extracting process equivalence is not merely a heuristic comparison, but a deterministic graph-theoretic reduction that guarantees the preservation of temporal properties.

2.3.4 Automata-Theoretic Verification and the SPOT Framework

Once LLM-generated code is abstracted into a state machine, cross-checking it against a business specification requires high-performance automata manipulation. The traditional approach to LTL model checking involves constructing a synchronous product between the system automaton and the negated property automaton. However, translating complex software workflows into finite-state structures frequently triggers the "state explosion problem," where memory requirements grow exponentially [29].

To mitigate this, contemporary verification pipelines leverage Transition-based Automata optimized through Binary Decision Diagrams (BDDs). Bryant [30] introduced BDDs as a canonical representation of boolean functions, fundamentally revolutionizing the memory efficiency of model checking. In the context of our framework, the translation and manipulation of these automata are grounded in the work of Duret-Lutz et al. [31], the primary architects of the SPOT library. SPOT provides state-of-the-art C++ implementations for LTL translation, emptiness checks, and Automata transformations. By relying on SPOT's BDD-backed dictionary (BuDDy), verification frameworks can lift dynamically typed Python implementations into a mathematical memory space, enabling the algorithmic intersection of the AI-generated code (`M_code`) against the specification (`M_spec`) without memory failure.

2.3.5 Implementation Space Clustering and Semantic Clone Detection

A unique challenge introduced by Generative AI is the multiplicity of outputs. A single prompt can create dozens of functionally equivalent but structurally divergent scripts. In traditional software engineering literature, identifying similar code structures is known as "clone detection."

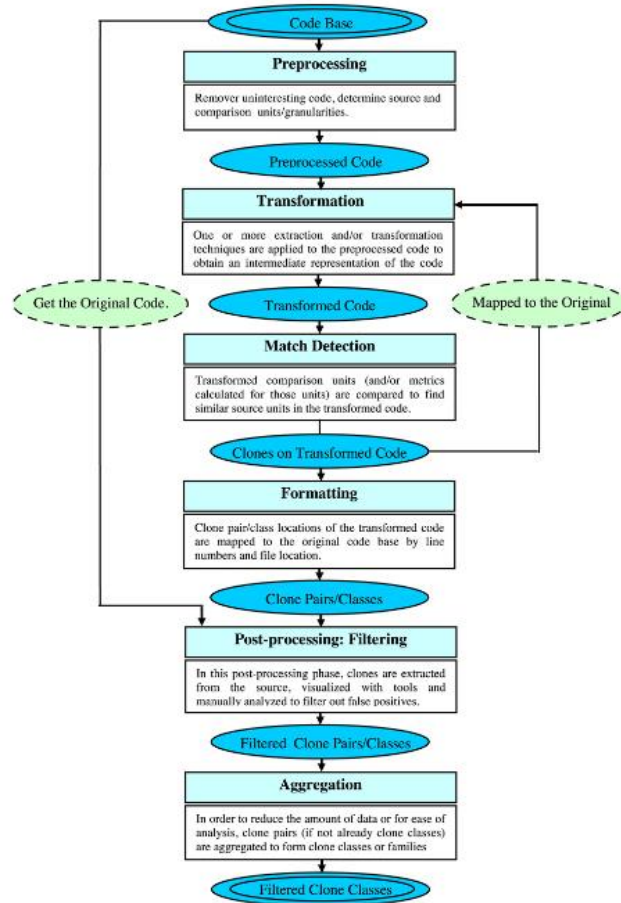


Figure 2.5 A generic clone detection process

However, classical clone detection tools rely heavily on token matching or Abstract Syntax Tree (AST) similarity [32], which fail when an LLM uses entirely different programmatic paradigms (e.g., iterative vs. recursive logic) to achieve the same business goal.

To address this, recent literature has shifted towards *semantic clustering* based on program execution traces and formal models. Jiang et al. [33] demonstrated that clustering algorithms applied to system dependency graphs can successfully group semantically equivalent implementations despite high syntactic variance. In the context of LLM auditing, applying implementation space clustering prior to final model checking provides a critical optimization strategy. By mathematically grouping process equivalent automata and extracting a single "Representative Automaton" for the cluster, the system significantly reduces computational overhead. Furthermore, automata, that fail to cluster with the majority serve as a primary indicator for anomaly detection, seamlessly flagging instances of LLM process drift or under-complexity [34].

2.4 Comparison of Approaches

Table 2.1 summarizes the key differences between our approach and existing methodologies across several critical dimensions.

Approach	BPMN Verification	Code Verification	Multiple Impl.	Confidence
BPMN-to-Petri-net [6]	Yes	No	No	N/A
LTL Patterns [7]	Yes	No	No	N/A
ClassEval [4]	No	Testing	No	Statistical
CompCert [15]	No	Full Proof	No	Absolute
Our Framework	Yes	Validation	Yes	Quantified

Table 2.1 - Comparison of Related Approaches

Our framework uniquely combines specification-derived and code-derived model construction with explicit translation validation and process-level equivalence analysis. Unlike traditional BPMN verification, we verify the implementation code

against the specification. Unlike LLM code evaluation, we provide formal verification rather than testing. And unlike verified compilation, we handle untrusted source code with quantified confidence rather than demanding full proof.

Chapter 3 - Technology Adapted

3.1 Python and the ast Module

Python 3.10+ serves as the primary implementation language for this framework. The choice of Python is motivated by several considerations. First, Python is the dominant target language for LLM code generation, with most benchmarks including ClassEval, HumanEval, and FLOW-BENCH providing Python implementations [4]. Second, Python's rich ecosystem of analysis tools, including the native AST module, provides powerful capabilities for program analysis without requiring external dependencies.

The AST (Abstract Syntax Tree) module is one of our approach. It provides deterministic, programmatic access to the syntactic structure of Python code, enabling the extraction of Control Flow Graphs (CFGs) with complete accuracy [18]. Unlike parsing approaches that rely on regular expressions or string manipulation, AST extraction is robust to formatting variations and comments, focusing solely on the program's semantic structure. The AST captures all control flow constructs, including conditions, loops, function calls, and exception handling, providing a complete representation of the program's control structure.

The AST-based extraction ensures that the control flow component of our intermediate representation is extracted with 100% accuracy, and there is no AI hallucination in this step. The structure of the model, including loops, conditions, and branches, matches the code exactly. This deterministic extraction provides a solid foundation for the subsequent semantic analysis stages.

3.2 Dynamic Tracing with sys.settrace

The sys.settrace facility provides a programmatic interface for instrumenting Python execution at runtime [19]. This capability is central to Role B's differential execution validation, enabling the collection of execution traces that capture the dynamic behavior of the program under test. By registering a trace function, our framework can record every line executed, every function call and return, and every exception raised, producing a comprehensive trace of the program's runtime behavior.

The primary advantage of dynamic tracing is that it captures the actual runtime behavior of the code, including Python's dynamic features that static analysis cannot fully resolve. For example, dynamic dispatch, monkey patching, and runtime code generation (through `eval` or `exec`) are transparent to runtime tracing but opaque to static analysis. By comparing traces collected from the original Python code against traces produced by a reference interpreter executing the extracted intermediate representation, we obtain strong evidence of semantic preservation [20].

Our framework uses randomized differential testing with `sys.settrace` to collect execution traces under varying inputs. The traces are then compared under a defined observation abstraction that maps concrete states to abstract observables, enabling meaningful comparison despite deliberate information loss in the abstraction process.

3.3 Z3 Theorem Prover for Symbolic Refinement

The Z3 theorem prover, developed by Microsoft Research, is an SMT (Satisfiability Modulo Theories) solver that provides automated reasoning about logical formulas over various theories including arithmetic, arrays, and bit vectors [21]. In our framework, Z3 serves as the engine for symbolic refinement checking, providing logical confidence in the correctness of the intermediate representation.

Concolic (concrete + symbolic) execution is employed to explore the program's behavior. The Python code is executed with symbolic inputs, and Z3 is used to check the feasibility of execution paths through the extracted intermediate representation. When a path through the IR corresponds to a feasible path in the original code (as determined by Z3), this provides logical evidence that the IR faithfully represents the code's behavior for that path. While this approach is bounded by the complexity of the program and the limitations of symbolic execution (particularly path explosion in loops), it provides a complementary verification modality to dynamic tracing [22].

The combination of dynamic tracing and symbolic refinement provides defense-in-depth validation where dynamic tracing offers statistical confidence through extensive testing, while symbolic refinement offers logical confidence for bounded program regions. When these two modalities agree, the combined confidence in the extraction correctness is significantly higher than either modality alone could provide.

3.4 SPOT Model Checking Library

SPOT (Spot is a Petri-net and Omega-automata Tool) is a state-of-the-art model checking library that provides efficient algorithms for LTL model checking, automata manipulation, and emptiness checking [23]. SPOT implements the automata-theoretic approach to model checking, which translates LTL formulas into Buechi automata, constructs the product with the system model, and checks for accepting runs.

Our framework leverages SPOT for two primary purposes. First, in Role A, SPOT is used to check that the LTLf properties extracted from the BPMN specification are satisfiable and to identify any contradictions in the property set. Second, in Role C, SPOT's automata-theoretic capabilities enable the comparison of specification-derived and code-derived automata through language inclusion checking and bisimulation computation. The divergence-sensitive stuttering bisimulation algorithm, which is central to our process-level equivalence definition, is implemented using SPOT's automata manipulation primitives [24].

SPOT was chosen over alternatives such as NuSMV for several reasons. Its programmatic C++ API enables tight integration with our Python-based pipeline through bindings, its performance on automata theoretic operations is among the best available, and its support for LTLf semantics aligns with our choice of finite-trace temporal logic.

3.5 NetworkX for Graph Analysis

NetworkX is a Python library for the creation, manipulation, and study of complex networks [25]. In our framework, NetworkX provides the graph data structures and algorithms used throughout the pipeline, from control flow graph construction to equivalence class computation.

The control flow graphs extracted from Python AST are represented as NetworkX directed graphs, enabling efficient computation of graph-theoretic properties including cyclomatic complexity, dominator trees, and strongly connected components. The BPMN process models are similarly represented as graphs, facilitating structural comparison between specification and implementation.

NetworkX's graph algorithms support the computation of equivalence classes through graph isomorphism and bisimulation checking on the abstracted automata.

The choice of NetworkX reflects a pragmatic balance between performance and ease of integration. While specialized graph libraries may offer better performance for very large graphs, NetworkX's seamless integration with Python's scientific computing ecosystem and its comprehensive algorithm library make it well-suited for our research prototype.

3.6 Version Control and Collaboration Tools

Given the modular architecture of the VibeCheck framework, maintaining strict version control and synchronized development is essential. **Git** was utilized as the distributed version control system to orchestrate the project's underlying codebase. Because the framework requires seamless integration between vastly different technological stacks, ranging from Python-based Abstract Syntax Tree (AST) parsing to low-level C++ automata manipulation via the SPOT library, Git provided the necessary isolated environments through feature branching. This allowed team members to independently develop, test, and refine their specific pipeline roles without introducing regressions or instability into the main execution pipeline.

To facilitate remote collaboration and repository management, **GitHub** was employed as the centralized hosting platform and continuous integration hub. Beyond standard code hosting, GitHub's robust pull request architecture was critical for conducting peer reviewed code integrations, particularly when merging the intermediate graph outputs of the Z3 verification engine with the final Stuttering Bisimulation clustering algorithms. Furthermore, GitHub's integrated issue tracking and project management boards were created to coordinate sprint tasks, and track debugging efforts.

Chapter 4 - Our Approach: Verified Translation Validation

4.1 Dual-Track Architecture Overview

The refined architecture implements a dual-track model construction and comparison framework that explicitly addresses the fundamental challenge of verifying LLM-generated code, which is how to establish confidence in the verification result when both the source code and the extraction pipeline are potentially unreliable. Rather than treating code-derived models as secondary or redundant, the framework positions both specification-derived and code-derived models as independent artifacts whose systematic comparison yields actionable verification results.

The pipeline proceeds through five interconnected stages:

S1: Specification Analysis. The BPMN 2.0 XML representation of the process specification is parsed to produce a property suite P with explicit coverage metrics and a specification-derived automaton M_{spec} .

S2: Code Execution and Trace Collection. The Python implementation is instrumented using `sys.settrace` to collect execution traces and construct a raw control-flow graph.

S3: Verified IR Extraction. The traces and CFG are processed through a multi-modal validation pipeline to produce a validated intermediate representation IR_{val} with an associated correctness certificate.

S4: Code-Derived Model Construction. The validated IR is lifted to a finite-state automaton, producing the code-derived automaton M_{code} .

S5: Equivalence Analysis and Classification. The specification-derived and code-derived automata are compared using process-level equivalence relations, producing a equivalence clusters for multiple implementations.

Figure 4.1 illustrates the dual-track architecture, showing how the specification track (top) and code track (bottom) converge in the equivalence analysis stage.

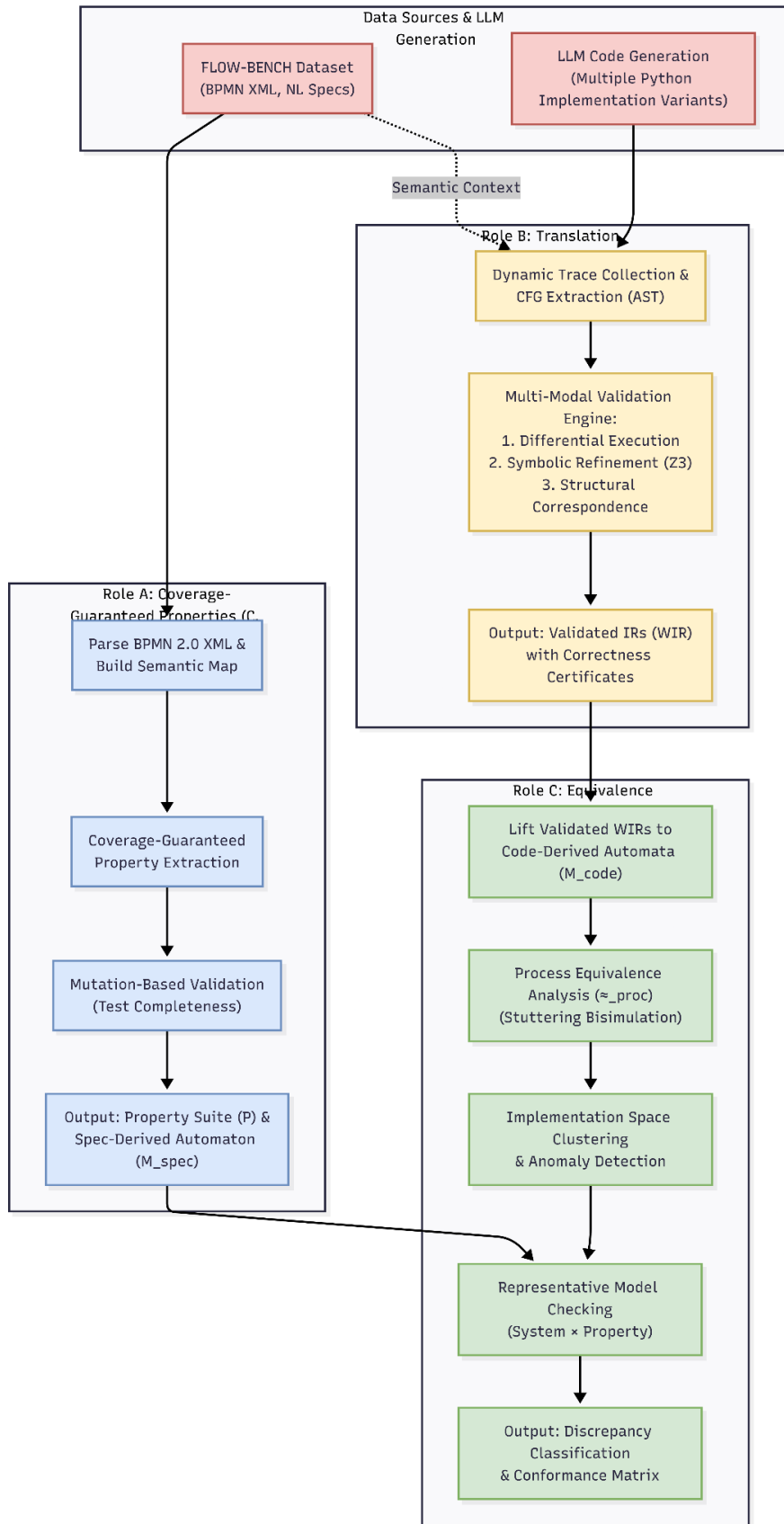


Figure 4.1: Dual-Track Verification Architecture

4.2 Module 01 : Property Extraction with Coverage Guarantees

Module 01 owns the specification-to-semantics track, transforming visual BPMN models into rigorous, analyzable formal representations. The primary novelty lies in the creation of a Self-Strengthening Formalization Framework that bridges the gap between informal human intent and AI-generated implementation. By introducing a Self-Auditing Specification Layer, the module ensures the formal "Ground Truth" is mathematically robust before the auditing process begins.

4.2.1 Structural Coverage Metrics for BPMN

Traditional BPMN-to-LTLf translation applies fixed pattern templates without systematic assurance that all relevant process constraints are captured. Here are three interconnected advances introduced.

Multi-Dimensional Structural Coverage: Adapts software testing principles to process models by quantifying node coverage (logical representation of all tasks), edge coverage (temporal constraints on all sequence flows), and path coverage (mathematical distinction of every feasible execution path).

Mutation-Based Sensitivity Validation: The system generates BPMN mutants—structurally modified variants representing plausible misinterpretations. It verifies that the property set "kills" (detects) each mutant; a surviving mutant indicates a semantic gap, transforming coverage from a static metric into an active validation mechanism.

Deterministic Inference of Implicit Logic: To handle informal or "lazy" modeling, the module automatically infers and formalizes negation guards (the "Implicit Else") for gateways. This ensures no logical "dead zones" exist where the auditor becomes blind to unintended code behavior.

Hierarchical Property Classification (P0, P1, P2): Extracted properties are categorized into a prioritized hierarchy: P0 (Critical) for safety and reachability, P1 (Structural) for control flow, and P2 (Quality) for best practices. This enables subtle conformance reporting rather than binary pass/fail results.

Sentinel Property and Negative Invariant Synthesis: Beyond describing what *should* happen, the system synthesizes Sentinel Properties. These describe forbidden states, creating a security perimeter against LLM "shortcuts."

Recursive Refinement & Threshold Enforcement: This is the framework's self-healing mechanism. If coverage falls below a predefined threshold (target = 0.95), the system triggers a diagnostic refinement loop. It identifies logical gaps, synthesizes new properties, and refuses to proceed to the audit phase until the specification is mathematically proven robust.

4.2.2 Property Schema and Template Instantiation

Module 01 operates on a controlled BPMN subset balancing expressiveness with analyzability: start and end events, tasks, exclusive gateways, parallel gateways, and bounded loops. For each BPMN pattern, properties are instantiated from predefined schemas using Fluent Linear Temporal Logic (FLTL), which extends LTL with fluents that hold between initiating and terminating events. Table 4.1 summarizes the pattern-to-property mapping.

BPMN Pattern	Category	FLTL Template	Parameters
Sequence A->B	Ordering	$G(\text{done}(A) \rightarrow F(\text{done}(B)))$	Task identifiers
XOR gateway	Coverage + Mutex	$G(\text{at}(G) \rightarrow F(\text{done}(B1) \text{ OR } \text{done}(B2)))$	Branch conditions
AND split/join	Parallel	$G(\text{at}(\text{join}) \rightarrow \text{AND } \text{done}(Bi))$	Branch IDs
Bounded loop	Termination	$G(\text{start}(L) \rightarrow F(\text{exit}(L) \text{ AND } \text{count} \leq n))$	Loop bound
Boundary Event	Exception	$G(\text{at}(A) \text{ AND } \text{trigger}(E) \rightarrow X(\text{NOT } (A) \text{ AND } F(\text{at}(H))))$	Error/Timer ID
Sentinel Guard	Safety	$G(\text{NOT}(A) \text{ until event}(B))$	Forbidden state

Table 4.1 - BPMN Pattern-to-Property Mapping

4.3 Module 02: Verified IR Extraction

Module 02 owns the code-to-semantics track with correctness evidence. The role develops a multi-modal validation architecture that provides quantitative confidence through complementary techniques with different failure modes.

4.3.1 Intermediate Representation Structure

The Workflow Intermediate Representation (WIR) is a JSON-structured labeled transition system with three layers. WIR-Core captures control flow constructs including basic blocks, conditional branches, bounded loops, and function calls. WIR-Data captures typed variables, assignments, and expressions. WIR-Proc captures process semantics including task boundaries (entry and exit events), data object references, and resource claims. WIR explicitly distinguishes control variables (affecting flow) from data variables (affecting only computation), enabling targeted abstraction.

4.3.2 Multi-Modal Translation Validation

The validation engine operates in three modes, each providing a different type of confidence evidence. Table 4.2 summarizes the validation modes.

Mode	Technique	Confidence Type	When Applied
V1	Differential Execution (sys.settrace)	Statistical	All IRs
V2	Symbolic Refinement (Z3)	Logical (bounded)	V1 passes
V3	Structural Correspondence	Syntactic	Fast pre-check

Table 4.2 - Validation Modes and Confidence Types

V1 (Differential Execution) performs randomized testing with sys.settrace, comparing execution traces of the original Python code against traces produced by a reference WIR interpreter under the observation abstraction. This provides statistical confidence that scales with test coverage.

V2 (Symbolic Refinement) performs concolic execution with Z3, checking path feasibility through the WIR against the original code. For bounded program regions, this provides logical confidence that is stronger than statistical evidence alone.

V3 (Structural Correspondence) performs fast syntactic checks matching CFG structure against WIR transitions. This provides a necessary condition for correctness and serves as a fast pre-check before more expensive validation modes.

4.4 Module 03: Equivalence Analysis

This is the comparison and reasoning layer, defining process-level equivalence relations, comparing specification-derived and code-derived models, classifying discrepancies, and analyzing implementation spaces. The core novelty is process-aware equivalence for implementation classification.

4.4.1 Three-Level Equivalence Hierarchy

There are three equivalence levels that form a hierarchy to evaluate LLM-generated code against the target business logic. Rather than a binary pass/fail, the engine evaluates code across three tiers to provide feedback. Table 4.3 defines each level.

Level	Definition	Preserves	Use Case
~fun	Same input-output behavior	Final results	Baseline
~trace	Same task entry/exit sequences	Action ordering	Detailed comparison
~proc	Same traces up to stuttering	Business observables	Conformance

Table 4.3 - Equivalence Level Definitions

The `approximate_proc` (process equivalence) relation is the key contribution. It collapses observationally equivalent implementations (such as different loop unrollings with the same task boundaries) while distinguishing process-divergent variants (such as multiplication versus repeated addition with different task structures). This equivalence is computed via stuttering bisimulation [24], which has been shown to appropriately capture process-level behavioral equivalence.

4.4.1.1 Evaluation Method

Equivalence Classification Accuracy: The percentage of correctly classified baseline scripts into the three tiers against manually annotated ground truth datasets.

To calculate the Equivalence Classification Accuracy, the framework first ingests a FLOW-BENCH dataset containing diverse LLM-generated workflow implementations that have been explicitly pre-labeled by human experts as isolated or clustered. Each generated script is systematically processed by the Equivalence Engine and autonomously categorized into one of the three hierarchical tiers (`approx_fun`, `approx_trace`, or `approx_proc`) based on its structural and behavioral conformance. The framework's automated classifications are then programmatically cross-referenced against the manually annotated labels to identify correct assignments, false positives, and misclassifications. The final accuracy score is computed as the exact ratio of correctly classified scripts to the total number of scripts analyzed, expressed as an overall percentage. Ultimately, this metric rigorously validates the engine's precision, proving that the automated clustering algorithm can reliably distinguish between acceptable AI-generated process variations and genuine structural anomalies without human intervention.

4.4.2 Divergence-Sensitive Stuttering Bisimulation Framework

To compute Process Equivalence (`approx_proc`), the framework implements a divergence-sensitive stuttering bisimulation algorithm.

Process-Aware Collapsing: This component mathematically collapses observationally equivalent implementations. For example, if an LLM generates different loop unrollings or redundant variable assignments that still share identical business task boundaries, the system merges these silent states.

Distinct Processes: On the other hand, the engine strictly distinguishes process different variants. For instance, if the LLM substitutes repeated addition with a multiplication paradigm that alters the underlying task structure, the bisimulation algorithm flags the structural divergence.

Computational Complexity and Scalability Analysis

In workflows which have high degrees of parallelism (AND gateways), the state space can expand factorially. To ensure the VibeCheck engine remains tractable for complex FLOW-BENCH workflows, we implement Parallel Gateway Abstraction, which treats parallel blocks as single abstract states, reducing complexity to linear.

4.4.3 Implementation Space Clustering

For a single BPMN specification with multiple Python implementations, the system executes a multi-implementation analysis pipeline.

Pairwise Computation & Grouping: The system takes in validated Workflow Intermediate Representations (WIRs) from Role B and computes pairwise scores. Implementations sharing the exact same process structure are mathematically grouped into a single cluster, vastly optimizing downstream processing.

Anomaly Isolation: Scripts that fail to cluster with the majority termed "singletons" are immediately isolated. This mechanism naturally filters out structural anomalies, such as LLM process drift or omitted constraints.

4.4.4 Representative Implementation

The final component bridges the implementation space with the formal business rules.

Cluster Verification: Rather than individually model-checking every generated script, the system extracts a single "Representative Implementation" from each valid cluster. This representative is formally verified against the property set extracted by Role A. The pass/fail verdict is then computationally propagated back to all members of that cluster.

Diagnostic Propagation: In the event of a failure, the system generates diagnostic explanations, mapping the formal mathematical counterexample back to the specific workflow discrepancy.

4.4.5 Summary of Multi-Implementation Analysis Pipeline

For a single BPMN specification with multiple Python implementations, the system executes a multi-step analysis pipeline. First, validated WIRs are extracted for each implementation using Role B. Second, pairwise approximate_proc is computed via

divergence-sensitive stuttering bisimulation. Third, implementations are clustered by equivalence, with singletons identified as anomalies. Fourth, representative implementations from each cluster are verified against the property set P from Role A. Finally, output results with diagnostic explanations identifying why each implementation passed or failed.

Chapter 5 - Analysis and Design

5.1 System Architecture Overview

The top-level system architecture comprises five interconnected modules (S1 through S5) that implement the dual-track verification pipeline. Figure 5.1 presents the conceptual architecture diagram showing the flow from inputs through processing stages to outputs.

The architecture follows a pipeline pattern with feedback loops. Data flows primarily from left to right, from raw inputs (BPMN XML and Python code) through processing stages to verification outputs. However, feedback paths enable iterative refinement: coverage gaps in S1 trigger additional property extraction; validation failures in S3 trigger abstraction refinement; and complexity thresholds in S5 trigger Bounded Equivalence Checking to maintain computational tractability during bisimulation.

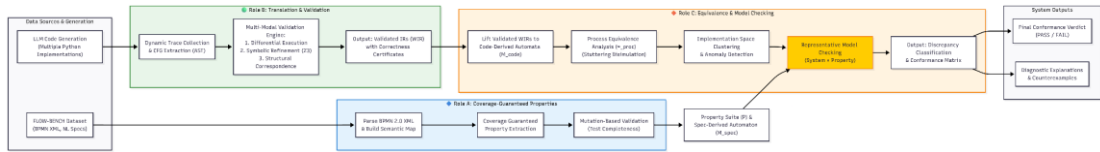


Figure 5.1: Top-Level System Architecture Diagram

The architecture explicitly supports multi-implementation analysis for a single BPMN specification, N Python implementations are processed through S2-S4 in parallel, producing N validated code-derived models that S5 analyzes for equivalence clustering and anomaly detection. This parallel processing is essential for analyzing the output of LLM generation, which typically produces multiple candidate implementations.

5.2 Module Descriptions

Stage	Input	Output	Role	Key Function
S1	BPMN XML + semantic map	Property suite P, M_spec	A	Parse BPMN, extract properties with coverage
S2	Python implementation	Execution traces, CFG	B	Instrument and collect runtime traces
S3	Traces + CFG + abstraction	Validated IR + certificate	B	Multi-modal validation with confidence
S4	Validated IR	Code-derived automaton	C	Lift IR to Automata representation
S5	M_spec, M_code, P, impl. Set	Conformance verdict	C	Equivalence analysis and clustering

Table 5.1 provides a detailed description of each module, including its inputs, outputs, owner role, and key functions.

5.3 Data Flow and Interactions

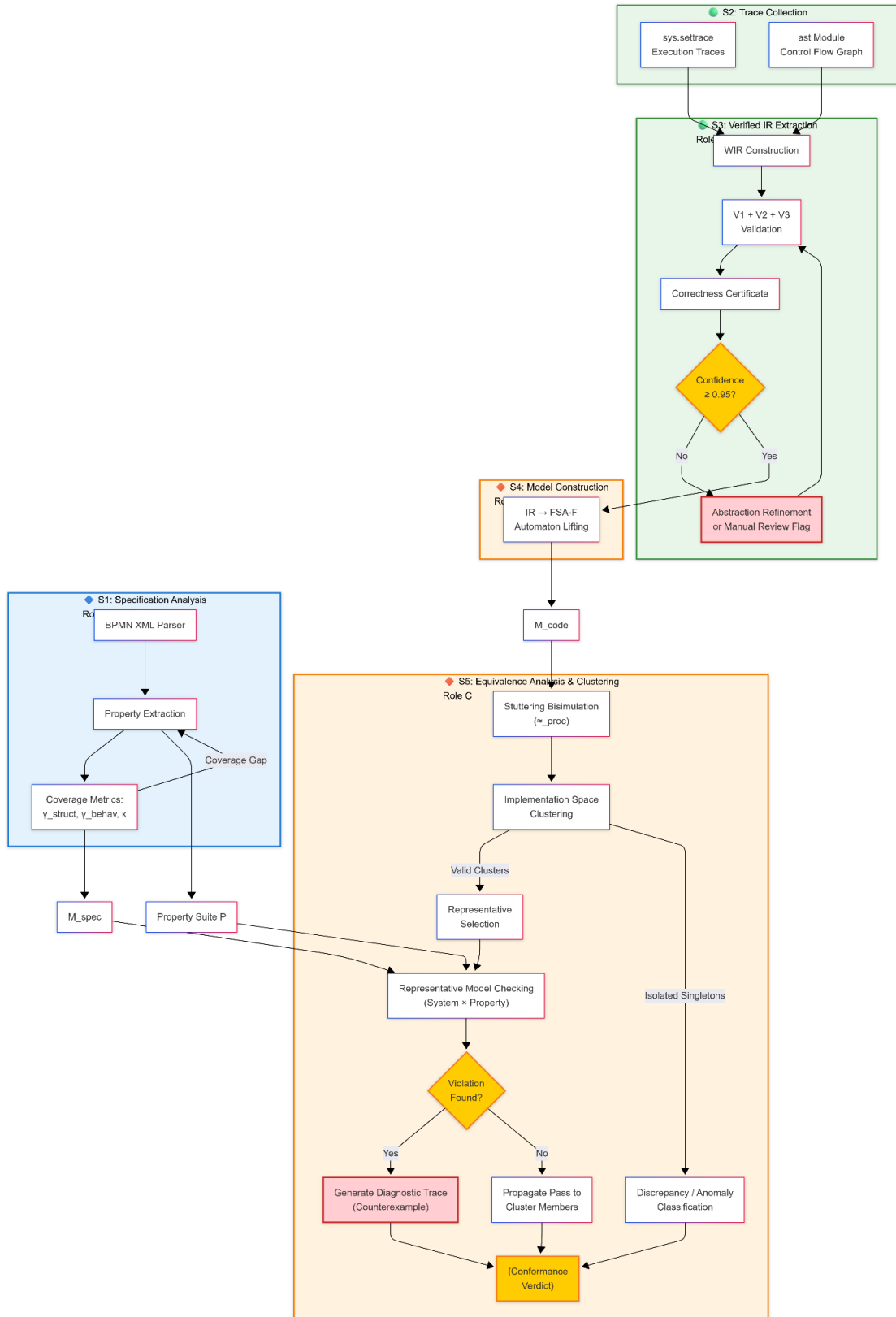


Figure 5.2 — Module Interaction and Data Flow

Figure 5.2 illustrates the module interactions and data flow within the system. The specification track (S1) and code track (S2-S4) operate largely independently until the final equivalence analysis stage (S5), where their outputs are systematically compared.

The interaction between S3 (Verified IR Extraction) and S4 (Code-Derived Model Construction) is particularly important. S3 produces not only the validated IR but also a correctness certificate containing the validation metrics (trace coverage, branch coverage, mismatch rate, refinement success, and combined confidence). S4 uses this certificate to determine whether the IR is sufficiently validated for downstream use. If the confidence is below the threshold (0.95), S4 may request additional validation from S3 or flag the implementation for manual review.

The interaction between S1 (Specification Analysis) and S5 (Equivalence Analysis) is similarly critical. S1 produces the property set P with coverage metrics, which S5 uses to verify representative implementations from each equivalence cluster. If a representative violates a critical (P0) property, the violation is propagated to all members of its equivalence cluster. This optimization avoids redundant property checking while preserving verification completeness.

Chapter 6 - Implementation

6.1 Dataset Preparation

The primary dataset for this research is FLOW-BENCH [11], a publicly available collection of 100+ workflow triplets consisting of natural language descriptions, BPMN 2.0 XML representations, and Python reference implementations. The dataset covers diverse domains including vending machine workflows, approval processes, inventory management, scheduling systems, and customer service processes. The BPMN coverage includes start and end events, tasks, XOR and AND gateways, bounded loops, and intermediate events, which aligns with the subset targeted by Role A's property extraction.

The dataset is partitioned into 80% for development and calibration (stratified by process complexity) and 20% held out for final evaluation. This ensures that the development set includes sufficient examples of each BPMN construct and control-flow pattern. Supplementary datasets include the Workflow Patterns Repository [24] for property template validation and the LTL Benchmark Suite for model checking infrastructure testing.

6.2 Implementation

6.2.1 Module 01

Early implementation has focused on engineering the Self-Auditing Specification Layer. The foundational infrastructure for Module 01 is now operational, shifting from simple XML parsing to a structured, validation-ready logic engine.

6.2.1.1 Implementation Details: Formal Extraction & Validation

The extraction pipeline ingests raw BPMN 2.0 XML and performs a multi-stage transformation into formal properties. A custom Semantic Mapping Engine traverses the XML tree, specifically targeting process nodes such as task, exclusiveGateway, and boundaryEvent.

As the parser traverses the model, it constructs a Semantic BPMN Map M_{spec} where nodes are mapped to atomic propositions and edges are mapped to temporal constraints. If the calculated Structural Coverage Y_{struct} falls below the 0.95 threshold, the engine triggers a Diagnostic Refinement Loop, auto-synthesizing missing properties before finalizing the specification.

6.2.1.2 Environment, Software Stack

Formal Verification and Mathematical Engines

- LTLf2Property (Logic Synthesis): A custom-integrated wrapper used by Module 01 to transform extracted FLTL templates into standard LTLf formulas compatible with the SPOT backend.
- XML Schema Validator: Employed in the Structural Coverage engine to ensure strict adherence to the BPMN 2.0 standard before property extraction begins, acting as the first security "Quality Gate."
- MutaBPMN Engine: A Python-based mutation testing utility developed for Module 01 to programmatically generate structural variants of BPMN XML files for K_{proxy} sensitivity validation.

6.2.2 Module 02

For module 02, this module is implemented using standard Python libraries. The current implementation successfully completes the static extraction, dynamic tracing, and initial differential validation phases.

6.2.2.1 Static Control Flow Graph (CFG) Extraction

AST-based CFG extractor has been implemented using Python's built-in ast module, producing control flow graphs for all Python implementations in the development set.

Implementation Details: The extraction algorithm ingests the raw Python scripts from the development dataset and parses them into a hierarchical AST. A custom NodeVisitor class was implemented to traverse the tree, specifically targeting control-flow nodes such as If, For, While, and Call. As the visitor traverses the tree, it

constructs a directed Control Flow Graph (CFG) where nodes represent executable statements and edges represent execution paths.

6.2.2.2 Symbolic Validation (Z3 Theorem Prover)

To ensure the extracted WIR does not contain hallucinated or mathematically impossible execution paths, an initial symbolic validation layer was implemented.

Implementation Details: The framework integrates the Microsoft Z3 Theorem Prover via its Python bindings. The validation engine iterates through every extracted edge condition in the CFG. The system autonomously identifies unknown variables within the LLM's guard conditions and declares them as Z3 Integers on the fly. The engine then mathematically evaluates the path.

6.2.3 Module 03

At the interim stage, the automaton lifting component has been prototyped, converting WIR representations to finite-state automata.

6.2.3.1 Environment, Software Stack

Formal Verification and Mathematical Engines

- SPOT Model Checking Library: The core engine utilized by Role C for automata generation, stuttering bisimulation, and the final synchronous product intersection. Accessed via official Python bindings.
- BuDDy (BDD Library): Integrated directly alongside SPOT to handle the algebraic compression of state transitions, actively mitigating the state-explosion problem during complex workflow verifications.

6.2.3.2 WIR to Labeled Transition System (LTS) Lifting

The core algorithm translates the JSON-based Workflow Intermediate Representation (WIR) from Role B into a formal Labeled Transition System (LTS), referred to as M_code. For the interim prototyping phase, this lifting is handled by a custom built Python abstraction engine, which establishes the formal mathematical graph prior to downstream C++ model checking integration.

The Lifting Algorithm proceeds in three procedural steps:

1. **State Initialization:** The algorithm parses the nodes array of the WIR. A unique mathematical state is instantiated for every node, and the `initial_state` (`S_0`) is established.
2. **Action Label Mapping:** To ensure traceability between the raw code and the formal model, the system iterates through the WIR, constructing a localized dictionary that maps node IDs to their semantic action labels.
3. **Formal Transition Construction:** The system iterates through the WIR edges, linking the instantiated states. It constructs formal transitions containing the source state, the target state, the symbolic guard condition, and the mapped action label.

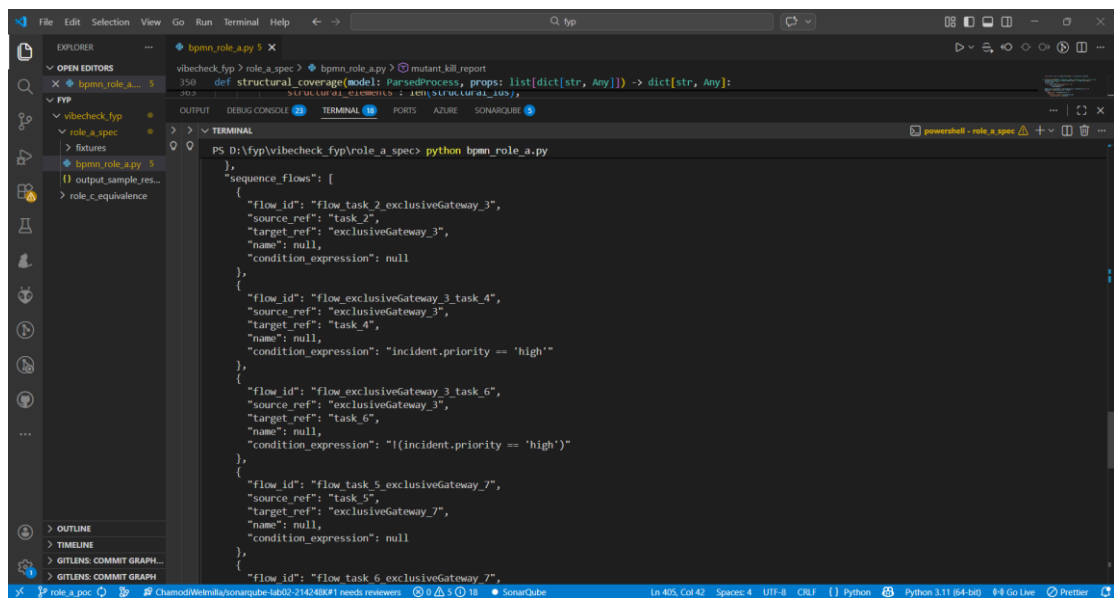
Chapter 7 - Discussion

7.1 Interim Test Results

7.1.1 Module 01: Semantic Mapping and Flow Extraction

To verify the foundational parsing capabilities of Module 01, the engine was tested against a multi-service integration workflow .

Test Execution: The extraction script was executed within the `role_a_spec` environment. The test targeted the engine's ability to parse complex BPMN XML sequence flows and correctly identify conditional branching logic (e.g., XOR gateway decisions) into a JSON-based intermediate representation.



```
def structural_coverage(model: ParsedProcess, props: list[dict[str, Any]]) -> dict[str, Any]:
    structural_coverage = {}
    for flow in model.sequence_flows:
        structural_coverage[flow.flow_id] = {
            "source_ref": flow.source_ref,
            "target_ref": flow.target_ref,
            "name": flow.name,
            "condition_expression": flow.condition_expression
        }
    return structural_coverage
```

Figure 7.1: Terminal output semantic mapping of BPMN sequence flows and conditional logic into a json intermediate representation.

Observed Results: As demonstrated in Figure 7.2, the engine successfully transformed the visual process model into a structured JSON map.

- **Relational Accuracy:** The system correctly identified source-to-target references.

- **Logic Extraction:** The engine successfully captured the boolean expressions required for the property suite, specifically extracting the high-priority conditions and its logical negation for the alternative path.

Conclusion: This test validates the **Semantic Mapping** stage of the formalization pipeline. It confirms that the framework can deterministically parse BPMN XML into a clean, machine-readable format, establishing the structural foundation necessary for the eventual generation of coverage-quantified properties.

7.1.2 Module 02: Static CFG Extraction

To verify the functionality of the static analysis module, the `ast_parser.py` engine was executed against a controlled set of reference implementations.

Test Execution: The extraction script was executed via the command line interface within the local `role_b_extraction` development environment. The engine was tasked with parsing `good_code.py` and `buggy_code.py` and translating their syntax into the Workflow Intermediate Representation (WIR).

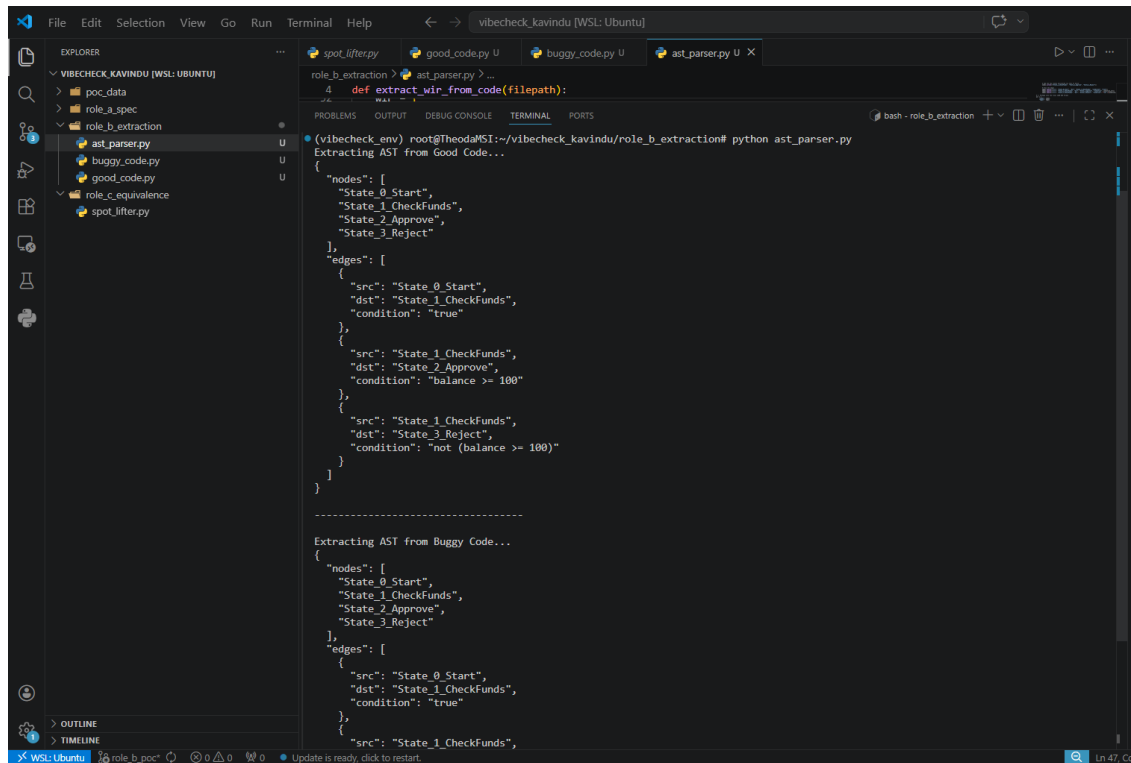
```

1  import ast
2  import json
3  import sys
4  import os
5  import re
6  import keyword
7  from z3 import * # Microsoft Z3 Theorem Prover
8
9  # PHASE 1: STATIC EXTRACTION (AST to Control Flow Graph)
10 class CFGExtractor(ast.NodeVisitor):
11     def __init__(self):
12         self.nodes = ["State_0_Start"]
13         self.edges = []
14         self.state_counter = 1
15         self.current_state = "State_0_Start"
16
17     def visit_If(self, node):
18         # Extract the condition as a string
19         condition_str = ast.unparse(node.test)
20
21         # Dynamically create generic states
22         true_state = f"State_{self.state_counter}_Iftrue"
23         false_state = f"State_{self.state_counter + 1}_Iffalse"
24         self.state_counter += 2
25
26         self.nodes.extend([true_state, false_state])
27
28         # Build the edges (Z3 requires capitalized 'Not' for negation in eval)
29         self.edges.append(("src": self.current_state, "dst": true_state, "condition": condition_str, "type": "true"))
30         self.edges.append(("src": self.current_state, "dst": false_state, "condition": f"!{condition_str}", "type": "false"))
31
32         # Store previous state, traverse inside the IF block, then restore
33         previous_state = self.current_state
34         self.current_state = true_state
35         self.generic_visit(node)
36         self.current_state = previous_state
37

```

Figure 7.2 Code Structure of AST parser

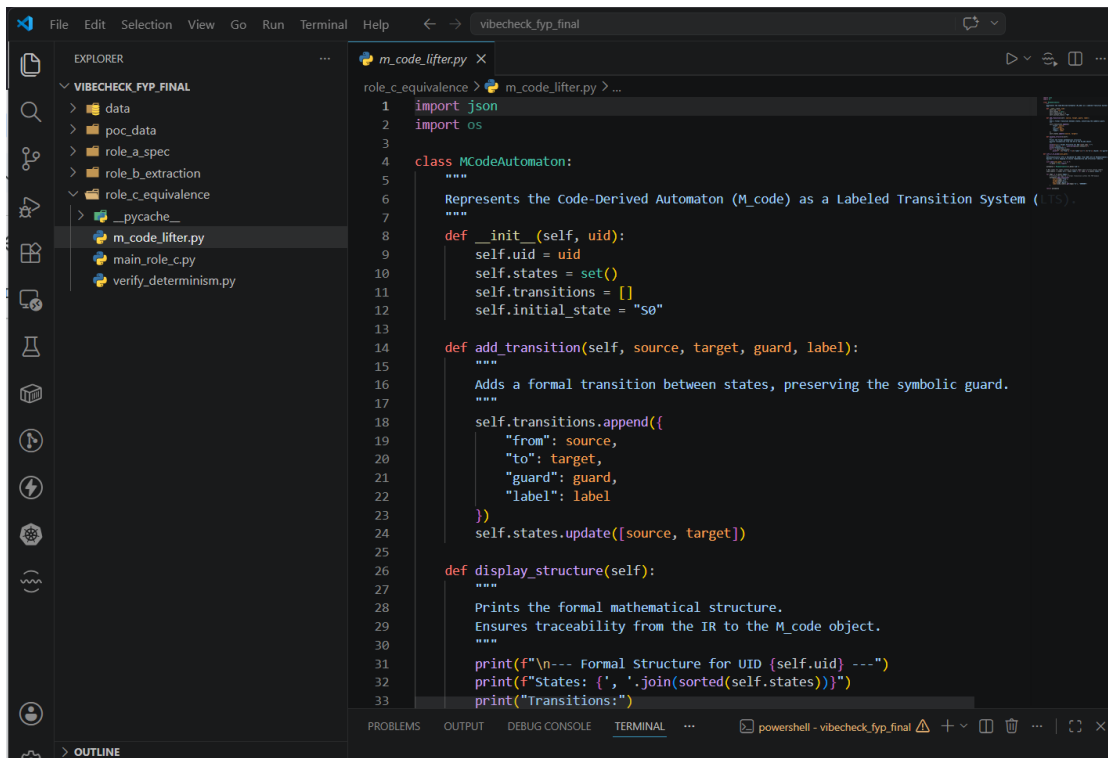
Observed Results: As demonstrated in Figure 7.2, the engine successfully ingested the raw Python scripts and outputted the exact JSON schema defined in the system architecture.



The screenshot shows a VS Code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project named 'VIBECHECK_KAVINDU [WSL: UBUNTU]' with several subdirectories and files. The terminal shows the output of running 'python ast_parser.py'. The output displays the JSON schema for the AST parser, which includes nodes and edges. The nodes are 'State_0_Start', 'State_1_CheckFunds', 'State_2_Approve', and 'State_3_Reject'. The edges are defined by conditions: 'condition: "true"' for the first edge, 'condition: "balance >= 100"' for the second edge, and 'condition: "not (balance >= 100)"' for the third edge.

```
def extract_wir_from_code(filepath):
    4
    5
    6
    7
    8
    9
    10
    11
    12
    13
    14
    15
    16
    17
    18
    19
    20
    21
    22
    23
    24
    25
    26
    27
    28
    29
    30
    31
    32
    33
    34
    35
    36
    37
    38
    39
    40
    41
    42
    43
    44
    45
    46
    47
    48
    49
    50
    51
    52
    53
    54
    55
    56
    57
    58
    59
    60
    61
    62
    63
    64
    65
    66
    67
    68
    69
    70
    71
    72
    73
    74
    75
    76
    77
    78
    79
    80
    81
    82
    83
    84
    85
    86
    87
    88
    89
    90
    91
    92
    93
    94
    95
    96
    97
    98
    99
    100
    101
    102
    103
    104
    105
    106
    107
    108
    109
    110
    111
    112
    113
    114
    115
    116
    117
    118
    119
    120
    121
    122
    123
    124
    125
    126
    127
    128
    129
    130
    131
    132
    133
    134
    135
    136
    137
    138
    139
    140
    141
    142
    143
    144
    145
    146
    147
    148
    149
    150
    151
    152
    153
    154
    155
    156
    157
    158
    159
    160
    161
    162
    163
    164
    165
    166
    167
    168
    169
    170
    171
    172
    173
    174
    175
    176
    177
    178
    179
    180
    181
    182
    183
    184
    185
    186
    187
    188
    189
    190
    191
    192
    193
    194
    195
    196
    197
    198
    199
    200
    201
    202
    203
    204
    205
    206
    207
    208
    209
    210
    211
    212
    213
    214
    215
    216
    217
    218
    219
    220
    221
    222
    223
    224
    225
    226
    227
    228
    229
    230
    231
    232
    233
    234
    235
    236
    237
    238
    239
    240
    241
    242
    243
    244
    245
    246
    247
    248
    249
    250
    251
    252
    253
    254
    255
    256
    257
    258
    259
    260
    261
    262
    263
    264
    265
    266
    267
    268
    269
    270
    271
    272
    273
    274
    275
    276
    277
    278
    279
    280
    281
    282
    283
    284
    285
    286
    287
    288
    289
    290
    291
    292
    293
    294
    295
    296
    297
    298
    299
    300
    301
    302
    303
    304
    305
    306
    307
    308
    309
    310
    311
    312
    313
    314
    315
    316
    317
    318
    319
    320
    321
    322
    323
    324
    325
    326
    327
    328
    329
    330
    331
    332
    333
    334
    335
    336
    337
    338
    339
    340
    341
    342
    343
    344
    345
    346
    347
    348
    349
    350
    351
    352
    353
    354
    355
    356
    357
    358
    359
    360
    361
    362
    363
    364
    365
    366
    367
    368
    369
    370
    371
    372
    373
    374
    375
    376
    377
    378
    379
    380
    381
    382
    383
    384
    385
    386
    387
    388
    389
    390
    391
    392
    393
    394
    395
    396
    397
    398
    399
    400
    401
    402
    403
    404
    405
    406
    407
    408
    409
    410
    411
    412
    413
    414
    415
    416
    417
    418
    419
    420
    421
    422
    423
    424
    425
    426
    427
    428
    429
    430
    431
    432
    433
    434
    435
    436
    437
    438
    439
    440
    441
    442
    443
    444
    445
    446
    447
    448
    449
    450
    451
    452
    453
    454
    455
    456
    457
    458
    459
    460
    461
    462
    463
    464
    465
    466
    467
    468
    469
    470
    471
    472
    473
    474
    475
    476
    477
    478
    479
    480
    481
    482
    483
    484
    485
    486
    487
    488
    489
    490
    491
    492
    493
    494
    495
    496
    497
    498
    499
    500
    501
    502
    503
    504
    505
    506
    507
    508
    509
    510
    511
    512
    513
    514
    515
    516
    517
    518
    519
    520
    521
    522
    523
    524
    525
    526
    527
    528
    529
    530
    531
    532
    533
    534
    535
    536
    537
    538
    539
    540
    541
    542
    543
    544
    545
    546
    547
    548
    549
    550
    551
    552
    553
    554
    555
    556
    557
    558
    559
    560
    561
    562
    563
    564
    565
    566
    567
    568
    569
    570
    571
    572
    573
    574
    575
    576
    577
    578
    579
    580
    581
    582
    583
    584
    585
    586
    587
    588
    589
    590
    591
    592
    593
    594
    595
    596
    597
    598
    599
    600
    601
    602
    603
    604
    605
    606
    607
    608
    609
    610
    611
    612
    613
    614
    615
    616
    617
    618
    619
    620
    621
    622
    623
    624
    625
    626
    627
    628
    629
    630
    631
    632
    633
    634
    635
    636
    637
    638
    639
    640
    641
    642
    643
    644
    645
    646
    647
    648
    649
    650
    651
    652
    653
    654
    655
    656
    657
    658
    659
    660
    661
    662
    663
    664
    665
    666
    667
    668
    669
    670
    671
    672
    673
    674
    675
    676
    677
    678
    679
    680
    681
    682
    683
    684
    685
    686
    687
    688
    689
    690
    691
    692
    693
    694
    695
    696
    697
    698
    699
    700
    701
    702
    703
    704
    705
    706
    707
    708
    709
    710
    711
    712
    713
    714
    715
    716
    717
    718
    719
    720
    721
    722
    723
    724
    725
    726
    727
    728
    729
    730
    731
    732
    733
    734
    735
    736
    737
    738
    739
    740
    741
    742
    743
    744
    745
    746
    747
    748
    749
    750
    751
    752
    753
    754
    755
    756
    757
    758
    759
    760
    761
    762
    763
    764
    765
    766
    767
    768
    769
    770
    771
    772
    773
    774
    775
    776
    777
    778
    779
    780
    781
    782
    783
    784
    785
    786
    787
    788
    789
    790
    791
    792
    793
    794
    795
    796
    797
    798
    799
    800
    801
    802
    803
    804
    805
    806
    807
    808
    809
    810
    811
    812
    813
    814
    815
    816
    817
    818
    819
    820
    821
    822
    823
    824
    825
    826
    827
    828
    829
    830
    831
    832
    833
    834
    835
    836
    837
    838
    839
    840
    841
    842
    843
    844
    845
    846
    847
    848
    849
    850
    851
    852
    853
    854
    855
    856
    857
    858
    859
    860
    861
    862
    863
    864
    865
    866
    867
    868
    869
    870
    871
    872
    873
    874
    875
    876
    877
    878
    879
    880
    881
    882
    883
    884
    885
    886
    887
    888
    889
    890
    891
    892
    893
    894
    895
    896
    897
    898
    899
    900
    901
    902
    903
    904
    905
    906
    907
    908
    909
    910
    911
    912
    913
    914
    915
    916
    917
    918
    919
    920
    921
    922
    923
    924
    925
    926
    927
    928
    929
    930
    931
    932
    933
    934
    935
    936
    937
    938
    939
    940
    941
    942
    943
    944
    945
    946
    947
    948
    949
    950
    951
    952
    953
    954
    955
    956
    957
    958
    959
    960
    961
    962
    963
    964
    965
    966
    967
    968
    969
    970
    971
    972
    973
    974
    975
    976
    977
    978
    979
    980
    981
    982
    983
    984
    985
    986
    987
    988
    989
    990
    991
    992
    993
    994
    995
    996
    997
    998
    999
    1000
    1001
    1002
    1003
    1004
    1005
    1006
    1007
    1008
    1009
    1010
    1011
    1012
    1013
    1014
    1015
    1016
    1017
    1018
    1019
    1020
    1021
    1022
    1023
    1024
    1025
    1026
    1027
    1028
    1029
    1030
    1031
    1032
    1033
    1034
    1035
    1036
    1037
    1038
    1039
    1040
    1041
    1042
    1043
    1044
    1045
    1046
    1047
    1048
    1049
    1050
    1051
    1052
    1053
    1054
    1055
    1056
    1057
    1058
    1059
    1060
    1061
    1062
    1063
    1064
    1065
    1066
    1067
    1068
    1069
    1070
    1071
    1072
    1073
    1074
    1075
    1076
    1077
    1078
    1079
    1080
    1081
    1082
    1083
    1084
    1085
    1086
    1087
    1088
    1089
    1090
    1091
    1092
    1093
    1094
    1095
    1096
    1097
    1098
    1099
    1100
    1101
    1102
    1103
    1104
    1105
    1106
    1107
    1108
    1109
    1110
    1111
    1112
    1113
    1114
    1115
    1116
    1117
    1118
    1119
    1120
    1121
    1122
    1123
    1124
    1125
    1126
    1127
    1128
    1129
    1130
    1131
    1132
    1133
    1134
    1135
    1136
    1137
    1138
    1139
    1140
    1141
    1142
    1143
    1144
    1145
    1146
    1147
    1148
    1149
    1150
    1151
    1152
    1153
    1154
    1155
    1156
    1157
    1158
    1159
    1160
    1161
    1162
    1163
    1164
    1165
    1166
    1167
    1168
    1169
    1170
    1171
    1172
    1173
    1174
    1175
    1176
    1177
    1178
    1179
    1180
    1181
    1182
    1183
    1184
    1185
    1186
    1187
    1188
    1189
    1190
    1191
    1192
    1193
    1194
    1195
    1196
    1197
    1198
    1199
    1200
    1201
    1202
    1203
    1204
    1205
    1206
    1207
    1208
    1209
    1210
    1211
    1212
    1213
    1214
    1215
    1216
    1217
    1218
    1219
    1220
    1221
    1222
    1223
    1224
    1225
    1226
    1227
    1228
    1229
    1230
    1231
    1232
    1233
    1234
    1235
    1236
    1237
    1238
    1239
    1240
    1241
    1242
    1243
    1244
    1245
    1246
    1247
    1248
    1249
    1250
    1251
    1252
    1253
    1254
    1255
    1256
    1257
    1258
    1259
    1260
    1261
    1262
    1263
    1264
    1265
    1266
    1267
    1268
    1269
    1270
    1271
    1272
    1273
    1274
    1275
    1276
    1277
    1278
    1279
    1280
    1281
    1282
    1283
    1284
    1285
    1286
    1287
    1288
    1289
    1290
    1291
    1292
    1293
    1294
    1295
    1296
    1297
    1298
    1299
    1300
    1301
    1302
    1303
    1304
    1305
    1306
    1307
    1308
    1309
    1310
    1311
    1312
    1313
    1314
    1315
    1316
    1317
    1318
    1319
    1320
    1321
    1322
    1323
    1324
    1325
    1326
    1327
    1328
    1329
    1330
    1331
    1332
    1333
    1334
    1335
    1336
    1337
    1338
    1339
    1340
    1341
    1342
    1343
    1344
    1345
    1346
    1347
    1348
    1349
    1350
    1351
    1352
    1353
    1354
    1355
    1356
    1357
    1358
    1359
    1360
    1361
    1362
    1363
    1364
    1365
    1366
    1367
    1368
    1369
    1370
    1371
    1372
    1373
    1374
    1375
    1376
    1377
    1378
    1379
    1380
    1381
    1382
    1383
    1384
    1385
    1386
    1387
    1388
    1389
    1390
    1391
    1392
    1393
    1394
    1395
    1396
    1397
    1398
    1399
    1400
    1401
    1402
    1403
    1404
    1405
    1406
    1407
    1408
    1409
    1410
    1411
    1412
    1413
    1414
    1415
    1416
    1417
    1418
    1419
    1420
    1421
    1422
    1423
    1424
    1425
    1426
    1427
    1428
    1429
    1430
    1431
    1432
    1433
    1434
    1435
    1436
    1437
    1438
    1439
    1440
    1441
    1442
    1443
    1444
    1445
    1446
    1447
    1448
    1449
    1450
    1451
    1452
    1453
    1454
    1455
    1456
    1457
    1458
    1459
    1460
    1461
    1462
    1463
    1464
    1465
    1466
    1467
    1468
    1469
    1470
    1471
    1472
    1473
    1474
    1475
    1476
    1477
    1478
    1479
    1480
    1481
    1482
    1483
    1484
    1485
    1486
    1487
    1488
    1489
    1490
    1491
    1492
    1493
    1494
    1495
    1496
    1497
    1498
    1499
    1500
    1501
    1502
    1503
    1504
    1505
    1506
    1507
    1508
    1509
    1510
    1511
    1512
    1513
    1514
    1515
    1516
    1517
    1518
    1519
    1520
    1521
    1522
    1523
    1524
    1525
    1526
    1527
    1528
    1529
    1530
    1531
    1532
    1533
    1534
    1535
    1536
    1537
    1538
    1539
    1540
    1541
    1542
    1543
    1544
    1545
    1546
    1547
    1548
    1549
    1550
    1551
    1552
    1553
    1554
    1555
    1556
    1557
    1558
    1559
    1560
    1561
    1562
    1563
    1564
    1565
    1566
    1567
    1568
    1569
    1570
    1571
    1572
    1573
    1574
    1575
    1576
    1577
    1578
    1579
    1580
    1581
    1582
    1583
    1584
    1585
    1586
    1587
    1588
    1589
    1590
    1591
    1592
    1593
    1594
    1595
    1596
    1597
    1598
    1599
    1600
    1601
    1602
    1603
    1604
    1605
    1606
    1607
    1608
    1609
    1610
    1611
    1612
    1613
    1614
    1615
    1616
    1617
    1618
    1619
    1620
    1621
    1622
    1623
    1624
    1625
    1626
    1627
    1628
    1629
    1630
    1631
    1632
    1633
    1634
    1635
    1636
    1637
    1638
    1639
    1640
    1641
    1642
    1643
    1644
    1645
    1646
    1647
    1648
    1649
    1650
    1651
    1652
    1653
    1654
    1655
    1656
    1657
    1658
    1659
    1660
    1661
    1662
    1663
    1664
    1665
    1666
    1667
    1668
    1669
    1670
    1671
    1672
    1673
    1674
    1675
    1676
    1677
    1678
    1679
    1680
    1681
    1682
    1683
    1684
    1685
    1686
    1687
    1688
    1689
    1690
    1691
    1692
    1693
    1694
    1695
    1696
    1697
    1698
    1699
    1700
    1701
    1702
    1703
    1704
    1705
    1706
    1707
    1708
    1709
    1710
    1711
    1712
    1713
    1714
    1715
    1716
    1717
    1718
    1719
    1720
    1721
    1722
    1723
    1724
    1725
    1726
    1727
    1728
    1729
    1730
    1731
    1732
    1733
    1734
    1735
    1736
    1737
    1738
    1739
    1740
    1741
    1742
    1743
    1744
    1745
    1746
    1747
    1748
    1749
    1750
    1751
    1752
    1753
    1754
    1755
    1756
    1757
    1758
    1759
    1760
    1761
    1762
    1763
    1764
    1765
    1766
    1767
    1768
    1769
    1770
    1771
    1772
    1773
    1774
    1775
    1776
    1777
    1778
    1779
    1780
    1781
    1782
    1783
    1784
    1785
    1786
    1787
    1788
    1789
    1790
    1791
    1792
    1793
    1794
    1795
    1796
    1797
    1798
    1799
    1800
    1801
    1802
    1803
    1804
    1805
    1806
    1807
    1808
    1809
    1810
    1811
    1812
    1813
    1814
    1815
    1816
    1817
    1818
    1819
    1820
    1821
    1822
    1823
    1824
    1825
    1826
    1827
    1828
    1829
    1830
    1831
    1832
    1833
    1834
    1835
    1836
    1837
    1838
    1839
    1840
    1841
    1842
    1843
    1844
    1845
    1846
    1847
    1848
    1849
    1850
    1851
    1852
    1853
    1854
    1855
    1856
    1857
    1858
    1859
    1860
    1861
    1862
    1863
    1864
    1865
    1866
    1867
    1868
    1869
    1870
    1871
    1872
    1873
    1874
    1875
    1876
    1
```

Test Execution: The lifting script was executed via the terminal and the code is in `main_role_c.py` file as shown below.



```
1 import json
2 import os
3
4 class MCodeAutomaton:
5     """
6     Represents the Code-Derived Automaton (M_code) as a Labeled Transition System (LTS).
7     """
8     def __init__(self, uid):
9         self.uid = uid
10        self.states = set()
11        self.transitions = []
12        self.initial_state = "s0"
13
14    def add_transition(self, source, target, guard, label):
15        """
16        Adds a formal transition between states, preserving the symbolic guard.
17        """
18        self.transitions.append({
19            "from": source,
20            "to": target,
21            "guard": guard,
22            "label": label
23        })
24        self.states.update([source, target])
25
26    def display_structure(self):
27        """
28        Prints the formal mathematical structure.
29        Ensures traceability from the IR to the M_code object.
30        """
31        print(f"\n--- Formal Structure for UID {self.uid} ---")
32        print(f"States: {' '.join(sorted(self.states))}")
33        print("Transitions:")
```

Figure 7.4 `main_role_c.py` file

Observed Results: As demonstrated in Figure 7.4, the engine successfully take the intermediate JSON files and instantiated the `M_code` formal structures.

- **State and Transition Instantiation:** The system successfully mapped the JSON nodes into distinct mathematical states ($S_0, S_1 \dots S_n$) and constructed the directional transitions


```

data > processed > irs > {} uid_1.json > ...
3   "flow_type": "linear",
4   "nodes": [
5     { "id": "S0", "label": "START" },
6     { "id": "S1", "label": "Jira_Issue_2_0_0_create_Issue" },
7     { "id": "S2", "label": "GitHub_Repository_3_0_0_create_Repository" },
8     { "id": "S3", "label": "END" }
9   ],
10  "edges": [
11    { "from": "S0", "to": "S1", "guard": "true" },
12    { "from": "S1", "to": "S2", "guard": "true" },
13    { "from": "S2", "to": "S3", "guard": "true" }
14  ]

```

```

PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu\root\vibecheck_fyp> wsl
Lifted UID 15: 5 states generated.
Lifted UID 7: 5 states generated.
Lifted UID 13: 7 states generated.
Lifted UID 12: 6 states generated.
(vibecheck_env) root@TheodaMSI:~/vibecheck_fyp/role_c_equivalence# python3 main_role_c.py

--- Formal Structure for UID 1 ---
States: S0, S1, S2, S3
Transitions:
S0 --(Jira_Issue_2_0_0_create_Issue)--> S1 [Guard: true]
S1 --(GitHub_Repository_3_0_0_create_Repository)--> S2 [Guard: true]
S2 --(END)--> S3 [Guard: true]

--- Formal Structure for UID 10 ---
States: S0, S1, S2, S3, S4, S5
Transitions:
S0 --(Amazon_S3_bucket_2_0_0_create_bucket)--> S1 [Guard: true]
S1 --(Box_File_3_0_0_retrievewithwhere_File)--> S2 [Guard: true]
S2 --(Amazon_S3_object_2_0_0_create_object)--> S3 [Guard: has_more_files]
S3 --(Amazon_S3_object_2_0_0_deletewithwhere_object)--> S4 [Guard: object.size > 100]
S4 --(Box_File_3_0_0_retrievewithwhere_File)--> S2 [Guard: true]
S5 --(Box_File_3_0_0_retrievewithwhere_File)--> S2 [Guard: object.size <= 100]
S2 --(END)--> S5 [Guard: !has_more_files]

--- Formal Structure for UID 11 ---

```

Figure 7.5 Execution output demonstrating the deterministic lifting of JSON intermediate representations into formal Labeled Transition Systems (LTS), preserving action labels and guard conditions.

Conclusion: This successful test validates the critical data hand-off between Role B and Role C. It proves that the framework can dynamically generate formal mathematical state machines (M_code) directly from LLM-generated code variants, establishing the necessary foundation for the downstream Stuttering Bisimulation and equivalence clustering algorithms.

7.2 Summary of the Report

This interim report has presented the design and early implementation of VibeCheck, a verified translation validation framework for LLM-generated workflow code. The framework addresses a critical gap in generative software engineering which is the absence of automated mechanisms to formally verify that LLM-generated Python implementations correctly realize their originating BPMN process specifications.

The key contributions presented in this report include,

- (1) a dual-track verification architecture that independently constructs specification-derived and code-derived formal models, enabling systematic comparison without circular reasoning.
- (2) a coverage-quantified property extraction methodology that transforms BPMN-to-property translation from an unmeasured process to one with explicit completeness metrics and active gap identification.
- (3) a multi-modal translation validation pipeline that replaces assumed IR extraction with validation providing certificates.
- (4) a process-level equivalence relation (`approximate_proc`) that distinguishes acceptable implementation variation. Which also can do meaningful classification of multiple implementations of the same specification.

7.3 Novelty and Differentiation

Our framework differs from existing approaches in four critical dimensions. First, unlike traditional BPMN verification tools such as Dijkman et al.'s BPMN-to-Petri-net translation [6], we verify the implementation code, not just the process model. This is essential when the code generator (the LLM) is untrusted. Second, unlike LLM code evaluation frameworks such as EvalPlus [13] that rely solely on testing, we provide formal verification that can prove properties hold for all possible executions within bounded state spaces. Third, unlike verified compilation approaches such as CompCert [15] that require trusted source programs and a prior proofs, we handle untrusted source code. Rather than attempting to prove the AI generator is flawless, we employ a deterministic, post-hoc model checking engine to guarantee the safety of individual generated artifacts. Fourth, we define process-level equivalence relations that capture business-meaningful behavioral similarity, going beyond functional equivalence to reason about task ordering, resource usage, and control-flow structure.

The novelty analysis from our refined design identifies three interconnected advances that together constitute a unique contribution. They are recursive refinement that enforce coverage-quantified property extraction with mutation-based validation, multi-modal translation validation with abstraction-parameterized correctness, and process-level equivalence for implementation space analysis. These advances are

practically significant, and they enable trustworthy verification of LLM-generated code at a time when generative AI deployment is accelerating across industry.

7.4 Present Plan on Further Work

The remaining work is organized into three phases. Phase 1 (Weeks 1-2) focuses on completing the core implementations: finalizing Role A's property extraction with full mutation-based validation, completing Role B's symbolic refinement integration with Z3, and implementing Role C's full process equivalence analysis completion and clustering pipeline. Phase 2 (Weeks 3-4) focuses on integration and initial evaluation: connecting all five stages of the pipeline, running them, and measuring coverage achievement on the FLOW-BENCH development set. Phase 3 (Weeks 5-6) focuses on evaluation results, performance benchmarking, and thesis writing.

References

- [1] Y. Li, H. Li, S. Zhang, M. Mao, J. Ma, and J. Zhang, "Hallucination risk in LLM-generated code: A survey," *IEEE Trans. Software Eng.*, 2024.
- [2] Z. Zhang et al., "The verification crisis in generative software engineering," in *Proc. 46th Int. Conf. Software Eng. (ICSE)*, 2024.
- [3] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [4] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, J. Zhang, and Z. Chen, "ClassEval: A manually-crafted benchmark for evaluating LLMs on class-level code generation," *arXiv preprint arXiv:2308.01861*, 2023.
- [5] A. Cimatti, E. Clarke, E. Giunchiglia, F. Giunchiglia, M. Pistore, M. Roveri, R. Sebastiani, and A. Tacchella, "NuSMV 2: An OpenSource tool for symbolic model checking," in *Proc. 14th Int. Conf. Computer Aided Verification (CAV)*, 2002.
- [6] R. Dijkman, M. Dumas, and C. Ouyang, "Semantics and analysis of business process models in BPMN," *Information and Software Technology*, vol. 50, no. 12, pp. 1281-1294, 2008.
- [7] S. Kherbouche and M. Ahmad, "Verification of business process models using temporal logic," in *Proc. 12th Int. Conf. Software Engineering and Applications*, 2017.
- [8] G. Aguirre, N. Aguirre, M. Frias, and M. Schaefer, "Translating YAWL to FSP to verify business process workflows," in *Proc. Brazilian Symposium on Formal Methods (SBMF)*, 2012.
- [9] G. De Giacomo and M. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proc. 23rd Int. Joint Conf. Artificial Intelligence (IJCAI)*, pp. 854-860, 2013.
- [10] G. De Giacomo, R. De Masellis, and M. Montali, "Reasoning on LTL on finite traces: Insensitivity to infiniteness," in *Proc. 28th AAAI Conf. Artificial Intelligence*, 2014.
- [11] E. Duesterwald, S. Huo, V. Isahagian, S. Kala, H. Khazraei, Y. Li, S. Malaika, and M. Vukovic, "FLOW-BENCH: Towards conversational generation of enterprise workflows," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing: Industry Track*, pp. 1426-1436, 2025.
- [12] Y. Li, Z. Wang, J. Li, J. Gao, and X. Zhang, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.
- [13] Y. Liu et al., "EvalPlus: Rigorous evaluation of LLM code generation," in *Proc. 37th Conf. Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] A. Backlund and L. Petersson, "Vending-Bench: A benchmark for long-term coherence of autonomous agents," *arXiv preprint arXiv:2502.15840*, 2025.
- [15] X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107-115, 2009.

- [16] L. Cordeiro and B. Fischer, "Verifying multi-threaded software using SMT-based context-bounded model checking," in Proc. 33rd Int. Conf. Software Engineering (ICSE), 2011.
- [17] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in Proc. 4th Int. Conf. Tools and Algorithms for the Construction and Analysis of Systems (TACAS), 1998.
- [18] Python Software Foundation, "ast - Abstract Syntax Trees," Python Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/ast.html>
- [19] Python Software Foundation, "sys.settrace," Python Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/sys.html#sys.settrace>
- [20] R. L. B. et al., "Behavioral abstraction for multi-language program analysis," in Proc. ACM SIGPLAN Int. Conf. Systems, Programming, Languages and Applications (SPLASH), 2023.
- [21] L. De Moura and N. Bjorner, "Z3: An efficient SMT solver," in Proc. 14th Int. Conf. Tools and Algorithms for the Construction and Analysis of Systems (TACAS), pp. 337-340, 2008.
- [22] C. Cadar and K. Sen, "Symbolic execution for software testing: Three decades later," Commun. ACM, vol. 56, no. 2, pp. 82-90, 2013.
- [23] A. Duret-Lutz et al., "From Spot 2.0 to Spot 2.10: What's new?," in Proc. 34th Int. Conf. Computer Aided Verification (CAV), 2022.
- [24] N. Aguirre, T. Maibaum, T. Meng, and N. Dutt, "Divergence-sensitive stuttering bisimulation for the verification of systems with conditional branching," in Proc. Brazilian Symposium on Formal Methods (SBMF), 2012.
- [25] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. 7th Python in Science Conf. (SciPy), pp. 11-15, 2008.
- [26] C. Baier and J.-P. Katoen, *Principles of Model Checking*. Cambridge, MA, USA: MIT Press, 2008.
- [27] R. Paige and R. E. Tarjan, "Three partition refinement algorithms," *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973-989, 1987.
- [28] J. F. Groote and F. Vaandrager, "An efficient algorithm for branching bisimulation and stuttering equivalence," in Proc. 17th International Colloquium on Automata, Languages, and Programming (ICALP), Warwick University, England, 1990, pp. 626-638.
- [29] E. M. Clarke, O. Grumberg, and D. Peled, *Model Checking*. Cambridge, MA, USA: MIT Press, 1999.
- [30] R. E. Bryant, "Graph-based algorithms for boolean function manipulation," *IEEE Transactions on Computers*, vol. 35, no. 8, pp. 677-691, Aug. 1986.
- [31] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, E. Renault, and L. Xu, "Spot 2.0 — A framework for LTL and ω -automata manipulation," in Proc. 14th International

Symposium on Automated Technology for Verification and Analysis (ATVA), Chiba, Japan, 2016, pp. 122–129.

[32] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," *Science of Computer Programming*, vol. 74, no. 7, pp. 470–495, 2009.

[33] M. Gabel, L. Jiang, and Z. Su, "Scalable detection of semantic clones," in *Proc. 30th International Conference on Software Engineering (ICSE)*, Leipzig, Germany, 2008, pp. 321–330.

[34] W. Wang, T. Guyer, C. Hastings, and A. Malony, "Anomaly detection in software execution using trace clustering," in *Proc. 19th IEEE International Conference on Automated Software Engineering (ASE)*, Linz, Austria, 2004, pp. 356–359.

[35] D. Binkley, "Source code analysis: A road map," in *Future of Software Engineering (FOSE '07)*, 2007.

Appendix A - Individual's Contribution to the Project

A.1 WelmillaCN - 214248K - Module 01: Property Extraction

Conducted an independent literature review exploring BPMN formalization, Linear Temporal Logic (LTL), and mutation-based verification, specifically focusing on the "semantic gap" between visual process models and verifiable implementation logic. The proposed solution introduces a Self-Auditing Specification Layer that utilizes a Recursive Refinement Loop to ensure the formal "Ground Truth" is mathematically robust before the auditing process begins.

For the design and interim implementation, I developed the foundational BPMN-to-FLTL (Fluent Linear Temporal Logic) extraction engine. I implemented a custom XML parsing pipeline using `xml.etree.ElementTree` to deterministically map visual BPMN constructs, including tasks, start/end events, and exclusive gateways, into a structured JSON intermediate representation. A core technical success in this phase was the successful extraction of Implicit Negation Guards for branching logic, ensuring every alternative path is formally represented. The most significant technical challenge during this phase was designing a scalable mapping protocol that preserves complex conditional expressions as atomic propositions within temporal formulas.

Currently, the engine successfully completes the semantic mapping and provides initial structural coverage Y_{Struct} metrics. This serves as the essential foundation for the next phase, where I will integrate the MutaBPMN engine to implement mutation-based sensitivity and complete the recursive refinement loop for automated property synthesis.

A.2 FERNANDO WKD - 214063M - Module 02: Verified IR Extraction

Conducted an independent literature review exploring software extraction techniques, static code analysis, and formal symbolic execution, specifically focusing on the limitations of traditional testing against LLM-generated code hallucinations. The proposed approach integrates Python's `ast` module for static structural extraction and

the Microsoft Z3 Theorem Prover (SMT solver) for dynamic state verification to construct a certified Workflow Intermediate Representation (WIR).

For the design and interim implementation, I defined the strict JSON schema for the WIR and completed the static extraction pipeline. I implemented a custom NodeVisitor class to traverse the Abstract Syntax Tree of the LLM-generated Python scripts, successfully extracting control-flow gateways (If, For, While) and generating directed Control Flow Graphs (CFG). Furthermore, I integrated the Z3 symbolic validation engine to mathematically verify the extracted paths. The most significant technical challenge during this phase was passing dynamically generated, unpredictable Python variables into the Z3.

In the next phase, I will expand this module to include dynamic execution tracing (sys.settrace) and differential execution to complete the multi-modal validation pipeline.

A.3 Hettiarachchi HTV - 214077J - Module 03: SPOT lifting IR

Conducted an independent literature review focusing on formal methods for process comparison, specifically exploring Bisimulation semantics, and Stuttering Equivalence. The proposed solution involves a hierarchical equivalence framework that utilizes Stuttering Bisimulation to bridge the gap between AI-generated code and deterministic BPMN specifications.

For the design and interim implementation, I formalized the Workflow Intermediate Representation (WIR) to Automaton translation logic and implemented the core Automaton Lifting Engine. This custom Python-based engine deterministically converts the JSON-based WIR into a formal Labeled Transition System (LTS). My implementation successfully maps semantic action labels and preserves symbolic guard conditions as formal transition constraints. The most significant technical challenge during this phase was ensuring structural fidelity during the lifting process; I addressed this by designing a 1-to-1 mapping protocol that translates WIR nodes into discrete mathematical states while maintaining the integrity.

Currently, the system successfully instantiates `M_code` objects and validates their structure. This serves as the essential foundation for the next phase, where I will integrate the SPOT model checking library to perform Binary Decision Diagram (BDD) encoding and implement the final Stuttering Bisimulation algorithm for implementation clustering.